ZF-Microglia-AI — Complete User Guide

For zebrafish confocal microscopy — step by step, from zero to microglia labels.

Table of Contents

---

1. What this plugin does

You have a confocal microscopy stack of a zebrafish brain. The image contains the brain you care about plus skin, tissue, and background surrounding it.

This plugin does two things, in order:

Step A — Skin Removal (Tab 1): Uses a trained AI model (MONAI 3D U-Net) to automatically detect and remove everything outside the brain, producing a clean `brain_only` image where only the cells of interest remain visible.

Step B — Label (Tab 2): From the cleaned image, automatically finds and labels each individual cell as a separately numbered 3D region, using one of two methods — **Cellpose-SAM Segmentation**, a fine-tuned AI foundation model and the recommended choice, or the **Pixel Classifier**, an older-technology, threshold-based fallback for machines with no GPU. The tab shows whichever one matches your Tab 1 output automatically; see Section 6a. Lets you sort, split, and edit labels before saving.

Step C — Analyse (Tab 3): Computes a comprehensive set of shape statistics for each labelled cell and exports them to a CSV file, with an optional AI-generated plain-language description per cell.

Beyond these three core steps, **Tab 4** launches GT annotation and MONAI/Cellpose-SAM training, and **Tab 5 — Sweeps & Utilities** (Section 9) collects every GT-verification sweep tool plus two related utilities in one place, so Tabs 1-3 stay focused on running the pipeline rather than tuning it.

> **Every numeric field with a slider next to it is directly editable** — click the number box and type an exact value instead of dragging the slider. Both stay in sync.

> **Every group box (the titled, bordered sections throughout all five tabs) is collapsible** — click its title checkbox to hide its contents, freeing up vertical space so groups further down become reachable. Useful on smaller screens where a tab has more sections than fit on screen at once. Collapsing doesn't discard anything — every field keeps its value, and re-expanding restores it exactly as it was.

> **Every group box also opens with a short description of what it actually does**, not just how to operate it — e.g. the Pixel Classifier and Cellpose-SAM Segmentation sections each explain their underlying pipeline (Gaussian smooth → threshold → union-find vs. `do_3D` → GMM → Krendl merge) before the click-instructions, not just after. Each tab itself opens with the same kind of short overview too, above its first group.

> **Each tab also scrolls independently** — if a tab is taller than your napari window even with some groups collapsed, a vertical scrollbar appears on the right edge of the panel so you can reach everything below the fold. Only vertical scrolling is enabled; the panel's width work means nothing should ever need to scroll sideways.

---

2. Installation

You need Python with napari already installed. Open a terminal and run:

```
pip install git+https://github.com/CTichy/ZF-Microglia-AI.git
```

All dependencies (PyTorch, MONAI, scikit-image, etc.) are installed automatically.

> **Mac with Apple Silicon (M1/M2/M3):** The plugin automatically uses your GPU via Metal (MPS). No extra steps needed.

> **Windows / Linux with NVIDIA GPU:** CUDA is detected and used automatically for Tab 1 inference and Tab 2 GPU-accelerated labelling (`cupy-cuda12x`, which has wheels for both platforms). Tab 3's *fastest* statistics path additionally uses `cucim` for GPU

> batch regionprops — this only has Linux wheels (RAPIDS/cuCIM has no native Windows build), so on Windows Tab 3 automatically falls back to a CPU-threaded path instead; nothing breaks, statistics just run somewhat slower. See Section 15 for details.

> **No GPU:** Works on CPU too, just slower (~30–60 minutes per stack for inference).

---

3. Getting the model files

The plugin needs up to **two** trained checkpoints, depending on which labelling method you plan to use. Neither is bundled in the plugin.

Suggested layout — not required (the plugin remembers whatever path you browse to), but a tidy default if you'd rather not decide where to put things:

```
Documents/
└── zf-microglia-ai-models/
    ├── MONAI/
    │   └── best_model_fullstack.pth
    └── Cellpose/
        └── <your checkpoint>
```

MONAI skin-removal model (required — Tab 1)

The AI model (~220 MB) that powers skin removal.

1. Download it:

   `https://cloud.technikum-wien.at/s/kYQ4qq3Jsn4xEyY`

2. Save the file `best_model_fullstack.pth` into `MONAI/` (or anywhere else easy to find).

3. Open the plugin, go to **Tab 1**, click the model `[...]` Browse button, and select the file — see "Model (.pth) — Browse button" under Section 5 below for the exact steps.

The plugin remembers the path — you only need to do this once per installation.

Cellpose-SAM checkpoint (optional — Tab 2, only if using Cellpose-SAM Segmentation)

Only needed if you plan to label cells with **Cellpose-SAM Segmentation** rather than the **Pixel Classifier** (see Section 6a). This is a project-specific fine-tuned Cellpose-SAM model (~580 MB), branch-weighted 3-fish checkpoint (`multi3_bw`, epoch 150).

1. Download it:

   `https://cloud.technikum-wien.at/s/eFBJepk9DakDxyb`

2. Save the file `cpsam_microglia_512_multi3_bw_epoch_0150` into `Cellpose/` (or anywhere else easy to find).

3. Open the plugin, go to **Tab 2**'s Cellpose-SAM Segmentation section, click the model **Browse `[...]`** button, and select the file. The path is remembered the same way as the MONAI model path.

If you don't have a checkpoint yet, use the **Pixel Classifier** instead — it needs no additional model file.

---

4. Opening the plugin in napari

1. Open a terminal and type `napari` to launch it.
2. In the napari menu bar, click **Plugins**.
3. Click **ZF-Microglia-ToolKit (ZF-Microglia-AI)**.
4. A panel appears on the right side with tabs: **Skin Remover**, **Create Labels**, **Statistics** (always visible — shows an explanatory hint in place of its controls until at least one Labels layer exists), **AI Tools** (always available — shows a disclaimer instead of hiding the tab if your GPU is missing or under the recommended 8GB, see Section 8), and **Sweeps & Utilities** (Section 9).

---

5. Tab 1 — Skin Remover

Open TIF / IMS file

Click this button to open your confocal stack (`.tif`, `.tiff`, or `.ims` format).

- All channels in the file are loaded as separate napari layers, each coloured differently:
  - Channel 0 → gray
  - Channel 1 → green
  - Channel 2 → magenta
  - Channel 3 → cyan
- Voxel size (physical scale in µm) is read automatically from the file metadata and applied to all layers.
- The folder and filename are remembered for automatic output file naming (see Section 10).

> **Important:** After loading, **click on the channel you want to process** in the Layers panel on the left. The plugin always runs on whichever image layer is currently selected (highlighted). For microglia, this is usually the green channel (ch1).

Load Labels layer (.tif)

Loads a previously saved labels `.tif` (e.g. from Save Labels or a GT correction) directly as a Labels layer, without going through Create Labels again. Useful for reopening a labels file for further correction, or bringing in a hand-corrected ground-truth file to compare against.

- Picks any `.tif`/`.tiff` file — the resulting layer is named `<active Image layer's own name>_labels`, the same convention Create Labels itself uses (e.g. `..._brain_only_ExtRm_labels`), not the source TIFF's own filename. Loading the same file again replaces that layer rather than creating a duplicate. If no Image layer is open at all, falls back to the file's own name.
- The new layer's **scale** is set to match the currently open stack's own physical (µm) voxel scale — the same one **Open TIF / IMS file** (above) applied to its own Image layers — so the loaded labels line up correctly in 3D space with whatever stack is already in the viewer. **Open the full stack first**, or the labels layer loads at

napari's own default (1, 1, 1) scale instead, with a warning in the status line.

Model (.pth) — Browse button `[...]`

Shows the path to the AI model file. If it says "— no model selected —":

1. Click the `[...]` button.
2. Navigate to where you saved the model file.
3. Select `best_model_fullstack.pth` and click Open.

The path is saved automatically to `~/.config/napari-zf-microglia-ai/config.json`. Next time you open the plugin, the model is already loaded.

Input info display

Below the model path, the plugin shows:

- The name and shape of the currently selected layer (e.g. `"NT54_ch1"  (300, 1024, 1024)  uint16`)
- The voxel dimensions: `Z=1.0000  Y=0.1740  X=0.1740 µm`
- The anisotropy ratio and the source of the scale information (from file metadata, from the layer scale, or default 1,1,1)

This is read-only — it updates automatically when you click a different layer.

MONAI Threshold

Range: 0.01 to 0.99 — **Default: 0.30**

The AI model outputs a probability map (0 = definitely not brain, 1 = definitely brain). This slider sets the cutoff: voxels above the threshold are classified as brain.

Value	Effect
0.20	More generous — includes uncertain areas; may keep some skin
0.25	**Recommended — validated best results (Nathalie)**
0.30	Previously documented default — superseded by 0.25

Value	Effect
0.50	Stricter — may cut into brain edges

> Post-processing (largest connected component + hole filling) cleans up most artefacts regardless of threshold. Keep it at 0.25 unless results look obviously wrong.

A read-only **Recommended MONAI Threshold** line sits underneath the slider — distinct from the slider's own live value, which stays freely editable. It only updates when **Verify MONAI Threshold / Erosion (GT Sweep)** (Tab 5, 9a) runs with its **"This is verified ground truth"** box ticked; moving the slider afterward to try something else never touches this line, so the sweep's own finding is never silently lost.

Erosion (vox)

Range: 0 to 15 voxels — **Default: 0**

After the brain mask is computed, this many voxels are stripped inward from the mask edge before applying it to `brain_only`. This removes a thin skin rim.

- **0:** No erosion — use the mask exactly as computed.
- **2–3:** Typical for zebrafish — removes a ~0.3–0.5 µm rim in XY or 2–3 µm in Z.

> The `brain_mask.tif` saved to disk is **always the un-eroded mask**. Erosion only affects `brain_only.tif`.

A read-only **Recommended Erosion** line sits underneath, same idea as MONAI Threshold's above — updated by either GT-verified sweep that tunes Erosion (Verify MONAI Threshold / Erosion, 9a, and Verify BG Threshold / Erosion, 9b, since both share this one Tab 1 slider).

Background (brain mode)

Four radio buttons controlling how background signal is handled after inference.

The background level is estimated automatically using the **mode** (most common intensity) of pixels inside the brain, computed from the result of inference. The mode represents the baseline scanner noise because background pixels vastly outnumber bright cell pixels.

Off

No background processing. `brain_only` = original volume × brain mask. Everything outside the brain is zeroed; everything inside is the original signal unchanged.

1 — Remove background outside brain (inference)

Removes background-level pixels only in the region **outside** the brain boundary. The brain interior is fully protected — nothing inside changes. Useful for cleaning up outer tissue while leaving the brain completely untouched.

- Requires BG Threshold (see below).

2 — Remove background globally (full stack) ⭐ Recommended before labelling

Removes all pixels across the **entire stack** (including inside the brain) whose intensity falls at or below the background threshold.

Result: Only the actual signal (bright microglia, stained cells) survives. Background becomes zero everywhere, leaving clean isolated blobs with empty space between them — exactly what the Create Labels algorithm needs.

Use this option before creating labels.

The saved filename gets the suffix `_NoBG`
(e.g. `NT54_ch1_brain_only_NoBG.tif`).

3 — Fill removed with random background

After skin removal, the region outside the brain is filled with **random noise** sampled from the actual background pixel distribution. The result looks like the original stack but with skin replaced by natural scanner noise — no hard black boundary at the brain edge.

- Uses Gaussian-filtered corner pixels as the noise pool (±2σ outlier removal) so the noise matches the real scanner texture.

- BG Threshold is not used in this mode.

The saved filename gets the suffix `_RndFill` (e.g. `NT54_ch1_brain_only_RndFill.tif`).

BG Threshold

Range: 0.00 to 2.00 — **Default: 0.50**

(Only active for background modes 1 and 2)

Fine-tunes the background removal threshold:

```
threshold = background_mode_value + BG_Threshold_offset
pixels ≤ threshold → removed (treated as background)
pixels  > threshold → kept (treated as signal)
```

Value	Effect
0.00	Threshold = exactly the mode — removes only confirmed background
0.50	Previously documented default
0.60	Previously documented "recommended for microglia" — superseded by 1.40
1.40	**Recommended for microglia labelling — validated best results (Nathalie)**
2.00 (max)	Aggressive — may remove dim signal from thin cell protrusions

> For microglia labelling, **1.40** typically produces the cleanest isolated blobs with good gaps between cells. If microglia are losing thin protrusions, lower the value.

A read-only **Recommended BG Threshold** line sits underneath — updated only by a GT-verified **Verify BG Threshold / Erosion (GT Sweep)** run (Tab 5, 9b).

Save checkboxes

- **Save brain_only.tif** (checked by default) — saves the brain-only volume with background removed
- **Save brain_mask.tif** (checked by default) — saves the binary mask as 0/255 uint8

Both files are saved in the output folder (see Section 10). The `brain_only` filename includes a background-mode suffix:

Mode	Suffix	Example filename
Off	(none)	`NT54_ch1_brain_only.tif`
1 — Exterior Removed	`_ExtRm`	`NT54_ch1_brain_only_ExtRm.tif`
2 — No Background	`_NoBG`	`NT54_ch1_brain_only_NoBG.tif`
3 — Random Fill	`_RndFill`	`NT54_ch1_brain_only_RndFill.tif`

Run Skin-Remover

Click to start processing. The button is greyed out while running; the status bar shows one summary line, and the small live-output box underneath streams progress from **both** stages the run actually goes through, not just the first one — MONAI's own sliding-window progress (one line per processed window) during brain segmentation, followed by the background/skin-removal step's own messages (background level detected, threshold applied, voxel counts removed or filled) once that starts. The same progress you'd see running each step from a terminal, previously invisible in the GUI for either. Tick **"Email me when done"** above the button first if you want a notification — see Section 9h for setup.

When complete, two new layers appear in napari:

- `*_brain_mask` — binary mask in cyan, semi-transparent
- `*_brain_only[suffix]` — the cleaned volume

Processing time: - NVIDIA GPU: ~30 seconds - Apple Silicon (MPS): ~5–10 minutes - CPU only: ~30–60 minutes

---

Verify MONAI Threshold / Erosion (GT Sweep) — moved to Section 9a, Tab 5 — Sweeps & Utilities. Recalibrates the Threshold/Erosion sliders above directly from a hand-corrected GT brain mask.

---

6. Tab 2 — Create Labels

> Before using this tab, run Tab 1 first, then click the resulting `brain_only` layer in the Layers panel to select it. Which section of Tab 2 appears depends on which background mode you used — see 6a below.

6a. Which tool is active — Pixel Classifier or Cellpose-SAM?

Use Cellpose-SAM Segmentation (6c) if you have a GPU. It's a fine-tuned AI foundation model and handles branching, overlapping, and faint microglia far better than classical thresholding — it's the labelling method every real result in this project has actually used. The **Pixel Classifier (6b)** is an older, simpler threshold-and-stitch tool kept around as an initial aid for machines with no GPU at all; treat it as a fallback, not a first choice, when a GPU is available.

Tab 2 shows **exactly one** of the two labelling methods below, chosen automatically from the active layer's filename suffix. Select a different layer in the Layers panel and Tab 2 switches live — no manual toggle needed.

Active layer ends in	Section shown	Produced by Tab 1 option
`_ExtRm`	**Cellpose-SAM Segmentation** (6c)	Option 1 — Remove background outside brain
`_NoBG`	**Pixel Classifier** (6b)	Option 2 — Remove background globally
`_RndFill`	*Neither* — this output is for presentation/ visualisation only	Option 3 — Fill removed with random background
anything else (e.g. the raw channel)	*Neither*, with a hint on what to select	—

So the choice is really made back in **Tab 1, Step 5**: pick **Option 1** if you plan to segment with Cellpose-SAM, or **Option 2** if you plan to use the Pixel Classifier.

The **Sort by / Resort Labels**, **Remove Debris**, **Split Label**, **Join Labels**, **Correct Label**, **Copy Label to Adjacent Slice**, **Correct Adjacent Labels**, and **Save Labels** tools (Section 6, further down) only appear once one of the two sections above is showing — with no `_ExtRm`/`_NoBG` layer selected, there's nothing yet to sort, clean up, split, or save. **Tab 3 — Statistics** takes a different approach: it stays visible regardless, showing an explanatory hint in place of its controls until at least one Labels layer exists in the viewer, so a first-time user can still discover the tab is there.

Common Settings — shared by both routes

Sits in its own box, above the Pixel Classifier/Cellpose-SAM sections and **always visible regardless of which one is currently active** — unlike everything documented under 6b/6c below, which only shows up when its own route is the one selected. These fields were deliberately pulled out of the route-specific sections: a value that's actually shared, or that looks shared but genuinely is not, is easy to lose track of if it only appears half the time. Cellpose-SAM's own Min size field is a deliberate exception — see 6c for why it lives in that route-specific section instead, not here.

Min volume (vox) — informative, not editable

Shown as plain text, not a slider — this value can't be typed or dragged directly. It's the smallest real cell volume ever confirmed by GT, shared by the Pixel Classifier's own volume filter and Cellpose-SAM's Safe-merge "already a whole cell" floor (`gt_min_from_labels()`, Krendl's own name for this quantity, and `min_volume_from_gt()` are literally the same computation) — an empirical fact measured from ground truth, not a knob to hand-tune. It only ever moves *down* (a new fish can only prove an even smaller real cell exists, never invalidate one already confirmed), and only from a **Tab 5 GT sweep** or a **GT-verified Generate Statistics run** (Tab 3) — never guessed, and never edited by hand. Starts at 7500 (the old default) until the first such measurement.

The actual tunable control for how strictly this floor is enforced is **Final min-size fraction**, below.

> Zebrafish microglia at 4dpf typically occupy 15,000–50,000 voxels at standard resolution.

Max volume (vox) — informative, not editable

Shown the same way as Min volume above, right below it — plain text, not a slider. The largest real cell volume ever confirmed by GT, tracked as its never-falling mirror: it only ever moves *up* (a new fish can only prove an even bigger real cell exists, never invalidate one already confirmed), and only from a **GT-verified Generate Statistics run** (Tab 3) — Tab 5 sweeps don't feed it, since none of them score a whole fish's every cell the way Statistics does. Reads "not yet measured" until the first such run.

Unlike Min volume, this doesn't drive any pipeline stage — nothing in either route deletes an oversized cell. It exists purely so `is_volume_outlier` in the Tab 3 Statistics CSV has something to flag against on the large side, the same way Min volume gives it something to flag against on the small side.

Min hole size (vox) — shared

Range: 0 upward — **Default: 0 (fill every enclosed gap, no minimum)**

Unlike Min/Max volume above, this one is a genuinely editable, **shared slider** used by both routes. A background region fully enclosed by signal — a "hole" — survives as real background only if its area is **at or above** this value; anything smaller is filled in as noise. Same idea as Min volume, just applied to gaps instead of whole objects, and named the same way on purpose: both name the size something must clear to be trusted as real, not the size at which it gets discarded.

The old behavior, unconditional hole-filling regardless of size, could silently erase real internal structure: a genuine gap inside a cell (debris exclusion, a real internal void) was treated identically to a single stray pixel of imaging noise, since neither route had any way to tell the two apart. Setting this above 0 draws that line explicitly, in both places it was missing:

- **Pixel Classifier** (Create Labels' own union-find pipeline): holes are filled **per 2D Z-slice**, since that pipeline builds labels slice by slice before stacking them into 3D objects.
- **Cellpose-SAM Segmentation**: Cellpose's own installed library fills holes in each predicted mask's full **3D** volume in one step (`cellpose/utils.py`'s `fill_holes_and_remove_small_masks()`, via `fill_voids.fill()`), also completely unconditionally. Since that code lives in the installed package, not this plugin, the fix is a

monkey-patch applied only for the duration of each Cellpose-SAM inference call — see `_make_capped_fill_holes()` in `_cellpose_seg.py` — rather than an edit to the installed library, which would be lost on every reinstall.

Value	When to use
0	**Default — fills every enclosed gap regardless of size (old behavior)**
Small (e.g. 20)	Only single-pixel noise gaps get filled; anything larger is preserved
Measured from GT	The smallest confirmed-real hole size seen in hand-corrected ground truth — see below

> Leave this at 0 unless you have actually seen real internal holes disappearing from your labels, or a GT sweep/GT-verified Statistics run has measured a recommended value from ground truth. There is no universal correct number — real GT checked during development showed a sharp split between 1–2 voxel gaps (near-certainly annotation noise) and 400+ voxel gaps (clearly real structure), with nothing in between, so a value anywhere in that gap works for that fish; a different fish may look different.
>
> Measured by every GT-sweep tool that has a GT labels volume available (both Pixel Classifier sweeps, 9b/9g, and the Cellprob/Large-contact sweep, 9c) **and** by a GT-verified Generate Statistics run (Tab 3) — all folded into the same never-rising floor.

Final min-size fraction — both routes

Range: 0.0 to 1.0 — **Default: 0.618 (the golden ratio, 1/φ)**

Unlike Min volume above, this one is a real, editable slider — it's the actual tunable control over how strictly the (non-editable) Min volume floor gets enforced. The actual small-object deletion cutoff **both** routes use is `this fraction × Min volume`, not Min volume itself — applied differently in each, since they have different pipeline shapes:

- **Cellpose-SAM (6c):** the very last stage, run after large-contact merge — any surviving cell below the cutoff is removed as a final safety net, regardless of how it survived every earlier stage. Nothing upstream is guaranteed to catch every debris object: GMM cleanup separates populations by the raw size distribution and can

still leave a gray-zone object standing, and safe-merge/large-contact only act when a nearby neighbor exists to merge into.
- **Pixel Classifier (6b):** applied directly as the volume filter's own cutoff, right after 3D objects are formed by union-find — this route has no merge/reattach stage of its own, so there's no later stage to hand a gray-zone object off to; the relaxed cutoff has to be the filter itself.

The golden ratio is the default for a specific reason, not decoration: it needs to be strict enough that a fragment genuinely that much smaller than the smallest real GT cell ever measured is a defensible "almost certainly not a real cell" cutoff, while staying lenient enough not to reject a legitimately smaller-than-average real cell the way using Min volume itself (fraction = 1.0) would. Set to `1.0` to recover the old, unrelaxed behavior (cutoff == Min volume exactly) on either route; `0.0` disables the Cellpose-SAM safety-net stage entirely and, on the Pixel Classifier, removes its volume filter's floor altogether (every non-zero object survives).

Soften label contours (sanding) after any label correction

Default: on — plus **Sanding sigma XY** and **Sanding sigma Z**, both in voxels, **default 0.7/0.7**.

A shared setting used by every tool that regenerates a label's shape from scratch: **Correct Label** (2D or 3D), **Correct Adjacent Labels**, and Cellpose-SAM Segmentation's own **Auto-correct** stage (6c). All of these rebuild a label voxel-by-voxel from an intensity threshold, which can leave contours blocky or spiky at the pixel scale. Sanding runs immediately after, on just the label(s) that tool touched: each one's own binary mask is 3D Gaussian-blurred (anisotropic — separate XY/Z sigma) and re-thresholded at 0.5, rounding off small jagged edges without meaningfully changing the cell's real shape or volume. Purely geometric — no image/intensity involved, unlike the correction itself.

Foreign-protected, same as every Correct Label tool: a neighboring label's already-claimed voxels can never be grown into, even where the blur would otherwise cross into them — so sanding one cell can never merge it into, or eat into, another. If a label's own shape shrinks to nothing under the blur (rare — usually means it was already tightly boxed in by neighbors) or loses all contact with its pre-sanding footprint, sanding is skipped for that label and the tool's status message says so; the correction itself is unaffected either way.

This is a **separate, independent setting from the Pixel Classifier's own Smooth σ XY/Z** (6b, default 1.5/3.0) — that pair decides whether raw blobs merge into one 3D object in the first place, before any labels exist. Sanding runs after labels already exist, only ever polishes one already-correct label's own edges, and is foreign-protected so it structurally cannot merge cells — which is why its default sigmas are much smaller: this is meant to be a light "sand the edges and spikes down" pass, not a reshape.

Uncheck this box to skip sanding entirely and get the older, unsoftened behavior back on all four tools at once.

6b. Pixel Classifier — Union-Find Labels

Fully self-contained: Gaussian smooth → threshold → per-slice 2D connected components → overlap-based union-find into 3D objects → volume filter → sequential renumber. Shown when the active layer ends in `_NoBG` (background removed everywhere, not just outside the brain) — needs no additional model file.

Smooth σ XY

Range: 0.0 to 5.0 — **Default: 1.0** — **Recommended: 1.5**

Controls the softness of blob contours **within each 2D slice** (the XY plane).

Gaussian smoothing is applied before thresholding each slice. This rounds jagged pixel edges and fills tiny holes within the same cross-section.

Value	Effect
0.0	No smoothing — raw pixel edges
1.5	**Recommended — solid, rounded blobs with preserved shape**
3.0+	Heavy — risk of merging nearby cells within the same slice

> Do not confuse with Smooth σ Z. They serve completely different purposes.

A read-only **Recommended Smooth σ XY** line sits underneath — updated only by a GT-verified **Verify Smooth σ XY / σ Z (GT Sweep)** run (Tab 5, 9g).

Smooth σ Z

Range: 0.0 to 5.0 — **Default: 0.5** — **Recommended: 3.0**

Controls **cross-slice connectivity** — how easily the algorithm links blobs in neighbouring Z slices into a single 3D object.

A microglia that disappears for 1–2 slices (due to low signal or a thin neck) and reappears will be correctly merged into one 3D object when σ Z is high enough.

> **Why σ Z = 3.0 while σ XY = 1.5?**
>
> Zebrafish confocal stacks are highly anisotropic: each Z slice is ~1 µm thick while each XY pixel is ~0.17 µm. So σ Z = 3.0 spans ~3 µm physically, while σ XY = 1.5 spans only ~0.26 µm.
>
> A microglia is typically 10–20 µm in diameter. Two microglia need to be closer than ~3 µm in Z for σ Z = 3.0 to risk merging them — which is uncommon in practice. This has been validated safe for zebrafish 4dpf microglia.

Value	Effect
0.0	No cross-slice smoothing — each slice fully independent
0.5	Minimal — only adjacent slices with strong overlap connected
3.0	**Recommended for zebrafish — bridges 1–3 slice gaps**
5.0+	Very aggressive — may link cells at different Z depths

A read-only **Recommended Smooth σ Z** line sits underneath, same as σ XY above.

Min overlap (%)

Range: 1 to 100 — **Default: 10%**

Two blobs in adjacent slices are recognised as the **same 3D cell** only if they share at least this fraction of the smaller blob's area:

```
overlap_ratio = shared_pixel_count / area_of_smaller_blob
if overlap_ratio ≥ min_overlap% → same object (linked)
```

- **Lower (5%):** Permissive — small touching fragments are linked.
- **Higher (30%):** Strict — only well-aligned blobs linked; isolated particles stay separate.
- **Start at 10%** and increase if too many fragments are joined, or decrease if cells are being cut across slices.

Create Labels

Click to run the 3D labelling algorithm. Processing runs in a background thread — the button is disabled until complete, and the small live-output box underneath shows the same per-stage messages the console gets (backend used, signal voxel count, blobs found/removed), instead of only landing in a terminal you may not have open. The volume filter's actual cutoff is `Final min-size fraction × Min volume` (Common Settings, 6a), not Min volume alone — see that field for why.

When done, a `*_labels` layer appears in napari with each detected cell shown in a different colour. The console prints how many labels were found.

Verify BG Threshold / Erosion (GT Sweep) — moved to Section 9b, Tab 5 — Sweeps & Utilities. Also measures the Min volume and Min hole size fields above directly from GT (running floors that only ever decrease) rather than leaving them guessed constants.

6c. Cellpose-SAM Segmentation

Shown when the active layer ends in `_ExtRm` (background removed only outside the brain — the interior is left intact for Cellpose-SAM to see). Runs `do_3D` Cellpose-SAM inference, then a 3-component-GMM cleanup pass, a Krendl safe-merge pass (rejoins sub-threshold fragments based on gap size and contact area), and a large-contact merge pass (catches cells accidentally split through a thick junction).

Min hole size, used by this route too, lives in the always-visible **Common Settings** box above (see 6a) rather than in this section — it stays visible there even while this section is hidden. **Min size** below is different: it's specific to this route, not shared, so it lives here instead.

Requires a Cellpose-SAM checkpoint — see Section 3. This is a project-specific fine-tuned model, not shipped with the plugin.

Model (.pt/checkpoint) — Browse button `[...]`

Browse to your trained Cellpose-SAM checkpoint file. The path is remembered across sessions, the same way the MONAI model path is remembered in Tab 1.

Min size (vox)

Range: 1 to 5000 — **Default: 15**

Cellpose-SAM's own early noise filter, applied right when raw instance masks are formed from the predicted flow field — previously hardcoded at 15 with no control anywhere in the plugin, now exposed here.

Not shared with Common Settings' Min volume — deliberately a different field, not the same value reused. Min volume is a route-agnostic GT-measured floor; nothing about it is specific to Cellpose-SAM. This field is: raw instance masks go through this small early filter, then **3-component GMM cleanup** and **Krendl safe-merge** — those two stages, not this field, make the real "is this debris, or a real fragment that should be reattached to a neighboring cell" decision, using the full size distribution and gap/contact geometry rather than one blunt cutoff. If Min size were raised anywhere near Min volume's range, real small fragments would be discarded here, before GMM cleanup or safe-merge ever got a chance to evaluate them for reattachment — quietly breaking the mechanism those two stages exist for. Leave this small; it only exists to catch prediction artifacts too small to be a fragment of anything.

Cellprob threshold

Range: -6.0 to 6.0 — **Default: -2.5**

Cellpose-SAM's own confidence cutoff for what counts as foreground (cell) vs. background. Lower (more negative) values are more permissive — they recover more of a cell's thin, dim protrusions but can also let in more noise.

A read-only **Recommended Cellprob threshold** line sits underneath — updated only by a GT-verified **Verify Cellprob / Large-contact (GT Sweep)** run (Tab 5, 9c).

Flow threshold

There is no Flow threshold field anywhere in this plugin, deliberately. In Cellpose generally, this parameter rejects predicted objects whose internal flow field doesn't self-consistently point back to a single centre — but Cellpose only applies that flow-error QC filter in 2D/stitch mode. Reading `cellpose/dynamics.py`'s `compute_masks()` shows the filter call sits inside `if not do_3D:`, and this plugin always runs `do_3D=True`, so the parameter has zero effect on any result this plugin produces. It used to appear as a slider that quietly did nothing; that was a real trap for anyone trying to tune it, so it was removed rather than merely documented. Internally, do_3D's function signature still requires a value, fixed at 0.4 and never exposed to the user.

Safe-merge max gap (vox)

Range: 0 to 20 — **Default: 2**

During the Krendl safe-merge pass, two fragments separated by a gap up to this many voxels are considered for merging into one cell (in addition to the contact-area check below).

Safe-merge min contact (vox)

Range: 0 to 200 — **Default: 10**

Minimum shared-boundary voxel count required between two fragments before the safe-merge pass will join them. Higher values require a more substantial touching surface before merging.

Safe-merge "already a whole cell" floor

No longer its own field here. The volume below which the safe-merge pass treats a fragment as *not yet* a whole cell and a candidate to merge into something else is exactly the same measurement as **Min volume** in Common Settings (6a) — the smallest true voxel volume ever confirmed in real GT — so this route now reads that shared field directly rather than keeping a second, separately-tracked copy of the identical number. Recalibrated by the **Verify BG Threshold / Erosion**, **Verify Smooth σ XY/σZ**, and **Verify Cellprob / Large-contact** sweeps

(Tab 5), and by Tab 3 Statistics whenever its “This is verified ground truth” checkbox is ticked — you shouldn’t normally need to set this by hand.

Large-contact merge (vox)

Range: 1 to 2000 — **Default: 20**

A second, separate merge pass for large blobs that got split apart through a thick junction (more contact area than the safe-merge pass alone would normally join). Raise this if large cells are still coming out fragmented; lower it if separate cells are being wrongly joined.

A read-only **Recommended Large-contact merge** line sits underneath, same as Cellprob threshold above.

Final min-size safety net

The last stage of this pipeline, run automatically after large-contact merge — see **Final min-size fraction** in Common Settings (6a) for the field itself and the golden-ratio reasoning behind its default. Nothing above it is guaranteed to remove every possible debris object: GMM cleanup separates populations by the raw size distribution and can still leave a gray-zone object standing, and safe-merge/large-contact only act when a nearby neighbor exists to merge into. This stage is the backstop underneath all of them, not a replacement for any one of them.

Run Cellpose-SAM Segmentation (button)

Click to start. `do_3D` inference is slow — it can take **hours** for a full-size fish — and runs in a background thread, so napari itself stays responsive while it works. The status line shows which pipeline stage is active (do_3D → GMM cleanup → Krendl safe-merge → large-contact merge), and the live-output box underneath streams Cellpose’s own internal progress during `do_3D` itself — normally invisible even from a terminal, since Cellpose only emits it through Python’s `logging` module and doesn’t configure a handler by default unless its own CLI is used. Tick **“Email me when done”** above the button first if you’d rather not watch — see Section 9h. When complete, a `*_labels` layer appears, exactly as with the Pixel Classifier.

> If this button errors with `No module named 'cellpose'`, install it in your environment: `pip install cellpose` (already listed in `environment.yml`/`environment-mac.yml` for fresh installs — see Section 15).

Auto-correct labels via contrast sweep after segmentation (checkbox, on by default)

When a full run finishes, this chains a second, fully automatic stage onto it — no GT, no manual contrast dragging:

1. **Calibrates the contrast threshold** that best reproduces the labels the run just produced, straight from the raw signal — the exact same self-referential engine as Calibrate Correct-Label Contrast (Section 9i), just run against this run's own output instead of a hand-picked sample set.
2. **Corrects every cell, whole-volume (3D)** at that threshold — the same engine as **Correct Label**'s **3D (whole cell, from centroid)** mode (below), run once per label automatically.
3. **Re-derives the boundary jointly, per slice, wherever two or more cells end up directly touching** — the same marker-seeded watershed as **Correct Adjacent Labels**, generalized to however many cells are actually touching there (not just pairs). This exists because step 2, run independently per cell, is order-dependent exactly at a shared boundary: whichever cell was corrected first wins the contested pixels by accident of processing order, not by anything about the real signal — this step replaces that with a real, symmetric split.
4. **Removes debris** once over the whole result (same golden-ratio floor as every other final-safety-net stage in this plugin).
5. **Softens every cell's contour (sanding)** — a fifth stage chained on afterward, gated by its own checkbox (Common Settings, above — see Soften label contours (sanding)), on by default. Purely geometric, runs on the full set of auto-corrected labels.

A cell that genuinely doesn't reach the calibrated threshold anywhere (rare, but possible for a very faint true positive) is skipped and left exactly as Cellpose-SAM originally produced it — this never aborts the run or erases a cell.

Saves the auto-corrected result to `<stem>_cp_krendl_ac.tif`, then — if sanding is enabled — saves the sanded result on top as `<stem>_cp_krendl_ac_snd.tif`, updating the `*_cellpose_labels` layer in place at each stage; `<stem>_cp.tif`/`<stem>_cp_krendl.tif` are unaffected — those stay the raw Krendl-pipeline output, unchanged. On-disk

naming is cumulative: each stage's filename is the previous one with one more suffix appended (see 9f's naming table), so the filename alone tells you which stages actually ran. The status line and log box report the calibrated `lo`, how many cells/touching-groups were corrected vs. skipped, how much debris was removed, and (if sanding ran) how many cells were softened. Untick the auto-correct checkbox to skip both this stage and sanding entirely and keep the older, single-pass behavior; untick only the sanding checkbox to keep auto-correct but skip the softening pass.

Re-run This Cell Only

Fixes one specific label without redoing the whole fish. Crops to that label's own bounding box (+ padding), re-runs `do_3D` plus the same GMM cleanup / Krendl safe-merge / large-contact merge / final-min-size safety net used by a full run — using this section's current settings, so nudge Cellprob or Flow iterations first if that's what needs to change for this cell — then splices the result back into the full label array in place of the old label. Requires the matching `<image>_cellpose_labels` layer to already exist (run the full segmentation once first).

Only crop-result pieces that actually overlap the original label's own footprint are kept — a neighboring cell caught by the padding and independently re-detected is discarded, not duplicated. If the re-run reveals more than one real cell in that crop, all of them are kept as new label IDs rather than forced back into one.

Label ID to re-run — type it directly, or click the cell in the viewer and use the **Use selected** button next to it.

Typical use: after a full run flags a porous/skeletonized cell (see the warning that appears after Run Cellpose-SAM Segmentation above), re-run just that one label at a stricter Cellprob (e.g. -0.3) instead of raising the setting for the whole fish — a fish-wide stricter threshold can clip real boundary pixels off cells that already segment fine, just to fix the minority that don't.

> **Tip:** select the volume layer (the `_ExtRm` image, not the labels layer) before clicking, if napari has auto-selected the labels layer from a previous action — the tool resolves the correct volume either way, but re-selecting the image layer directly is the more obvious path.

Verify Cellprob / Large-contact (GT Sweep) — moved to Section 9c, Tab 5 — Sweeps & Utilities. Also recalibrates the shared Min volume field (Common Settings, 6a) from GT — Safe-merge's floor and Min volume are the same number now, not two separately-tracked copies of it.

Build GT-Correction Package — moved to Section 9f, Tab 5 — Sweeps & Utilities.

Sort by / Reverse order / Resort Labels

After creating (or loading) labels, you can renumber them by a criterion of your choice.

Sort by dropdown:

Option	Meaning	Default order
Size	Number of voxels	Largest = label 1
Centroid Z	Z coordinate of centre	Smallest Z = label 1
Centroid Y	Y coordinate of centre	Smallest Y = label 1
Centroid X	X coordinate of centre	Smallest X = label 1

Reverse order checkbox — inverts the ordering (e.g. smallest = label 1 for Size).

Click **Resort Labels** to apply. The active Labels layer is renumbered 1…N in the chosen order, in place. This is useful for consistent numbering across samples or for matching cells to a reference atlas.

Remove Debris

Manual edits in napari — deleting a whole label because it turned out to be misclassified skin, splitting one, painting part of one away — can leave small disconnected fragments behind that never went through Create Labels' own volume filter (which only ran once, before the edit). Click **Remove Debris** and the active Labels layer, exactly as it currently stands, is swept for anything smaller than `Final min-size fraction × Min volume` (Common Settings, 6a) — the same golden-ratio-relaxed cutoff Create Labels' own filter and Cellpose-SAM's final safety-net stage already use — and every one found is zeroed out.

Evaluated per spatially-connected fragment, not per label ID. Erasing most of a wrongly-segmented label by hand (painting over it) commonly leaves several small, disconnected leftover pieces still carrying that *same* original ID — and their combined voxel count can look large enough to survive even though every individual piece is genuine debris on its own. This tool checks each disconnected piece's own size, not the sum across everything still tagged with that ID, so those leftovers are correctly caught.

Works on labels from either route. Surviving labels' IDs are left exactly as they were — this only removes, it never renumbers (use **Resort Labels** above afterward if you also want a clean 1…N sequence). A label ID with one real surviving piece and one small disconnected leftover keeps its ID on the real piece; only the leftover is zeroed.

Split Label

Splits a single merged label (a blob where two or more cells are stuck together) into separate parts using a 3D watershed algorithm.

The watershed approach finds the **thinnest neck** connecting two large volumes and cuts there — it does not use a simple distance threshold or Euclidean splitting.

Target label

The label number of the blob you want to split. You can type it directly, or:

1. In the napari viewer, hover over the blob and read the label number shown in the status bar.
2. Click the blob to select it in the Labels layer.
3. Click **Use selected** — the label number is filled in automatically.

Use selected

Reads the currently selected label from the active napari Labels layer and fills it into the Target label spinner. Click the blob in napari first, then click this button.

Split mode

3D (whole label) (default) — splits the entire 3D blob, exactly as described above.

2D (current slice only) — restricts the whole operation to the single Z-slice you're currently viewing: crops to that label's footprint on just that slice, runs the same watershed pipeline in 2D, and splices the result back into only that slice. Every other slice of the label is left completely untouched, and the new piece exists only on that slice — it doesn't extend into neighboring slices the way a 3D split's parts would.

Use 2D mode when two things only touch on **one** cross-section rather than forming a genuine 3D neck — e.g. real signal happening to graze an unrelated skin-residue fragment right at that particular slice, with no real connection above or below it. In that case a 3D split either can't find a cut there at all (there's no true 3D saddle to watershed on) or ends up cutting somewhere unrelated on a different slice instead. Navigate to the problem slice in napari before clicking Split Label.

Split into N parts

Range: 2 to 10 — **Default: 2**

How many separate pieces the blob should be divided into. The algorithm searches for the N largest sub-volumes (separated at their thinnest necks) and cuts between them.

> If the blob genuinely has only one major volume (no neck), splitting may fail or produce uneven results. Increase Smooth σ or use a lower Min distance if that happens.

Smooth σ (Split)

Range: 0.0 to 3.0 — **Default: 1.0**

Gaussian smoothing applied to the distance transform before searching for peaks. Higher values smooth out the distance map, making the algorithm more robust to surface noise but less sensitive to subtle necks.

- **0.5–1.0:** Suitable for most cases.
- **1.5–2.0:** Use if the split point jumps around — smoother distance field = more stable result.
- **0.0:** No smoothing — very sensitive to surface texture.

Min distance

Range: 1 to 30 voxels — **Default: 5**

Minimum voxel distance required between accepted seed peaks. If two candidate peaks are closer than this, only the stronger one is kept.

- **Too high:** The two centres of a closely-packed double-blob may be rejected as "too close" → fewer than N peaks found → error.
- **Too low:** Surface noise peaks may be accepted as separate centres → wrong split point.
- **5 voxels** works well for microglia-sized cells.

Split Label (button)

Click to run. The original blob is replaced in-place:

- The original label number is kept for the **first** part (the largest sub-volume).
- New label numbers (`max_existing + 1`, `max_existing + 2`, …) are assigned to the remaining parts.

The cut is **interface-only**: exactly the voxels at the boundary between parts are removed, creating a 1-voxel gap. The outer surface of each part is not touched — thin protrusions are preserved.

If the algorithm cannot find N distinct sub-volumes, an error message is shown. Try reducing Smooth σ or Min distance.

Join Labels

The inverse of Split Label: merges Label B into Label A when one cell was wrongly cut into two pieces (e.g. a thin process fooled the segmenter into treating it as a neck). A single, fast, whole-volume operation — no bounding-box crop or GPU path needed, since nothing here depends on shape or geometry.

Label A (keep) / **Label B (merge into A)** — type each label number directly, or click the corresponding fragment in the viewer and use that field's own **Use selected** button (click one fragment, grab it into A; click the other, grab it into B).

Click **Join Labels**. Every voxel currently carrying Label B becomes Label A instead; Label A's ID survives, Label B's ID disappears entirely. Works on labels from either route.

Correct Label

Regenerates a label's shape from the raw signal layer's own live contrast display — useful when a particular contrast window traces a cell's true silhouette more accurately than the existing label does. Works in **2D** (one slice) or **3D** (the whole cell), via the **Correction mode** dropdown.

How it works (both modes): dial the signal (`_ExtRm`/`_NoBG`/raw channel) Image layer's contrast limits in napari until the cell's outline looks right by eye — a narrow window (e.g. displaying only values from ~100 up) usually works best, since it saturates the display into a clean silhouette rather than showing a soft gradient.
`Calibrate Correct-Label Contrast (from Cellpose-SAM)` in Tab 5 can find this value automatically instead of by eye — see below. Then, without changing anything else, run this tool: it reads that same contrast window straight off the layer and uses everything **at or above** the lower limit as signal — the upper limit is just napari's own display-saturation ceiling, not a boundary on what still counts as real signal, so it isn't used to exclude anything.

Fills the whole padded box, not just what already connects to the label. Every pixel at or above the threshold within the bounding box (+ padding) becomes part of the corrected label, whether or not it's touching (even diagonally) the label's existing shape — real cell boundaries are often irregular or branchy, and signal that's visibly part of the cell to the eye shouldn't be discarded just because it doesn't already connect at this exact threshold. The only pixels ever excluded are ones already claimed by a *different* label (never touched, absorbed, or grown into) and anything below the threshold. Any small leftover fragment this produces that isn't really part of the cell is exactly what **Remove Debris** (above) is for — in 3D mode, the tool also runs that cleanup automatically at the end (see below).

Signal layer — pick the Image layer whose current contrast window should be used (populated automatically from whatever Image layers are open).

Label to correct — type it directly, or click the label in the viewer and use **Use selected**. Accepts any positive label ID, or **-1** for the Protect Skin as Label sentinel itself — useful for re-trimming skin's own territory by hand after it's been created. 0 and any other negative value are rejected outright (there's never a real label there).

Bbox padding (px) — default 15, matching the padding convention used elsewhere in this plugin (e.g. crop extraction). Each corrected slice only looks within this label's own bounding box (+ padding) on that slice, not the whole image.

Correction mode

- **2D (current slice only)** — corrects only the slice you're currently viewing.
- **3D (whole cell)** — corrects the entire cell. **Not a bounding box at any point.** This loops over 2D areas, one Z slice at a time, each with its OWN local neighborhood — the label's own CURRENT footprint on that one slice, padded — reusing the same already-correct 2D tools directly: no other label present in that local area → plain single-label 2D correction (exactly Correct Label's own 2D mode); another label present → the same joint watershed split Correct Adjacent Labels uses, scoped to this label alone, so the local boundary against a genuinely adjacent cell gets properly re-derived right there too, not just excluded as an immovable wall. The label's own original Z range is corrected this way outright (it already exists on every one of those slices), **walked mid-outward, not in plain Z order**: the walk starts at the range's own upper-middle slice and settles outward in two halves — middle down to the bottom, then middle+1 up to the top. Before each slice's own correction runs, its watershed seed is first topped up with whichever of its immediate neighbors (one slice up, one slice down) fall within the label's own original range: a neighbor the walk has already reached contributes its own just-corrected shape; one not yet reached still contributes its own original, pre-walk shape — real signal, useful context either way, refined by its own correction once the walk gets there. This union only ever ADDS territory into empty background or skin (never onto a different real label's own pixels), purely as a bigger starting marker for that slice's own watershed — the final shape is still entirely re-derived from the real intensity threshold there, so a neighbor's shape can never force-keep a pixel that isn't also real signal on THIS slice. The practical effect: a good correction on one slice actively helps its neighbor's own correction, and a single spuriously undersized original slice — ordinary Cellpose-

SAM prediction noise, not necessarily a real taper — gets topped up by its neighbor's own shape as an integrated part of the walk, instead of needing a separate pre-pass to guard against it (an earlier version of this tool did exactly that as a first step, before this mechanism replaced it). A genuinely tapering cell still comes out correctly smaller slice by slice regardless — only a truly undersized *original* slice gets rescued, never a real, deliberately narrower one, since the final shape always comes from real signal, never from the seed alone. Growth beyond the original range works the same way but one-directional: copying the current slice's own just-corrected shape onto the next Z first (foreign-protected, same as Copy Label to Adjacent Slice), then running the same local correction there — the moment that finds nothing to support the copied position, the copy is undone and growth in that direction stops, the true edge found. This is also the tool's own answer to "is there real signal at the very end of the cell the segmentation missed" — it keeps extending outward, one slice at a time, for as long as real signal keeps supporting the new position. A hard cap (the same value as **Bbox padding** unless auto-grow overrides it) still bounds how far this can ever extend regardless, so a real, different, genuinely-touching structure can't pull growth along indefinitely just because it happens to overlap slice-to-slice.

(An earlier version instead sized ONE Y/X window from the label's entire 3D extent and applied that same fixed window to every slice being corrected or walked through. That turned out to be a real design flaw, not just an inefficiency: a window sized from the whole extent can be far bigger, on any one slice, than where the cell actually sits there — and since a padded working area's whole-box fill has no connectivity requirement (see the note under Correct Label), it would happily claim whatever OTHER real, not-yet-labeled signal — skin residue, a different unlabeled structure — happened to also fall inside that oversized window on that slice, then potentially pull the walk further along following THAT signal's own extent on the next slice. A per-slice, always locally-anchored area — re-derived fresh from wherever the label actually currently sits, every single slice — can't do this: it can only ever reach as far as the padding from the label's own real, already-verified position one slice ago.)

Because growth only ever adds a slice after confirming real continuation, and the original range is always corrected in place, there's no separate "trim leftover pixels beyond where growth stopped" step needed either.

A debris-cleanup pass then runs automatically (the same golden-ratio-relaxed floor Cellpose-SAM's own final safety net uses) to remove any small disconnected leftover — scoped to **only this label**, so it can never affect any other cell in the fish.

A report appears below the button listing every slice the correction touched, how many debris pixels were removed, and — for each of those slices — any *other* label whose pixels either directly **touch** the correction or merely sit **nearby** (inside the same padded working region, without necessarily touching). This is a visibility check, not a safety gate: a neighboring label's own pixels are never absorbed or overwritten either way (structurally impossible — the correction can only ever claim pixels that were background or already this label) — the report just flags close calls worth a manual look.

Click **Correct Label**. A neighboring label's own pixels are never touched, absorbed, or grown into — even if they sit inside the same padded box and share the exact same intensity range as the label being corrected. If the chosen contrast window leaves no signal at all anywhere in the padded box, the tool refuses to apply rather than silently emptying the label — adjust the window and try again.

If Soften label contours (sanding) (Common Settings) is checked — the default — the corrected label is sanded immediately afterward, in either mode: same foreign-label protection, purely geometric, rounds off blocky edges left by the intensity-threshold correction. The status line reports whether it was applied.

Auto-grow until signal clears the border

Default: off. A fixed padding can cut off real signal that genuinely extends further than expected — the correction fills the padded area as far as it can, but has no way to tell "the cell truly ends here" from "the area ran out of room." This checkbox catches that: if the corrected result touches the edge of its own padded working area, it automatically retries with a bigger one, repeating until the result stops touching the edge or the iteration limit is hit. Reaching the true edge of the volume is never flagged — there's nothing more to grow into there anyway.

- **Growth step (px)** — how much the padding grows each retry (default 15, same as the base padding default).
- **Max growth iterations** — hard cap on retries (default 5), so this can never grow unbounded. If it's still touching the border after the

cap, the tool stops and tells you rather than continuing to grow — a real signal that the cell may extend even further, worth a manual look. **In 3D mode, the report names exactly which slice(s) are still touching the edge** — go correct those individually (a bigger pad on just that slice via Correct Label's own 2D mode, or by hand) instead of guessing which part of the cell needs more room. The same slice list is shown even without auto-grow, whenever a plain 3D correction's own report says `touched_border`.

In 2D mode, if the growing box starts overlapping a different label, that label is automatically folded into the correction instead of being encroached on — the tool switches to the same joint, foreign-protected correction Correct Adjacent Labels uses (generalized to any number of labels, not just two), so growth can never silently eat into a neighbor's territory. The status line reports if the group grew this way. There's only one shared padding value here, since 2D corrects a single slice — nothing to grow "per slice."

In 3D mode, growth is PER SLICE, not a whole-cell retry. Since Correct Label's own 3D mode already loops over 2D local areas rather than one bounding box, each slice's own padding grows independently, only for the slice(s) that actually need it — a cell with 40 well-behaved slices and one long branch reaching further on a single slice only ever regrows THAT one slice, at whatever bigger pad it needs. Every other slice keeps its own already-correct result and its own (smaller) pad completely untouched. This is both cheaper (no reason to redo 39 already-fine slices just because one needed more room) and converges far more easily — one slice needing extra room no longer means the whole cell has to be reprocessed at that bigger pad before it can be judged converged, the way an earlier version of this tool worked. The report lists exactly which slice(s) ended up needing a bigger pad and what they grew to, plus every OTHER label the final correction ended up near, for visibility. A final debris-cleanup pass (same golden-ratio floor as everywhere else) runs once at the end.

Sanding (if enabled) runs on every label the final correction reported as nearby, not just the one you originally targeted.

Keep re-running until the shape stabilizes (3D only)

Default: off. Independent of auto-grow above — usable with it, without it, or instead of it. Every 3D correction doesn't just reshape the label you're correcting: wherever it finds a genuinely adjacent label (another real cell, or a Protect Skin as Label neighbor), the joint watershed split it runs there also updates THAT label's own boundary

— seeded from each label's CURRENT shape at the time. Feeding one pass's result back in as the next pass's starting point lets the boundary settle a little closer to a mutually-consistent position each time, since each watershed reseeds from a slightly different (just-updated) marker than the pass before — repeated enough times, this converges toward a genuine fixed point where re-running produces the identical result again, rather than stopping after just the first (possibly still-settling) boundary placement.

This runs PER SLICE, exactly like auto-grow does — not a whole-cell retry. Each slice keeps re-correcting itself, feeding its own result back in as its own next attempt's starting point, until THAT slice's footprint matches what the immediately preceding attempt on that same slice produced — or **Max stability passes** (default 10) is reached, whichever comes first. A slice with no adjacent label at all still needs a minimum of 2 attempts to *confirm* nothing is changing (there's no baseline to compare against on the very first attempt), but stays cheap; only a slice genuinely still settling against a real neighbor keeps going beyond that. Every other slice is left completely alone once it's confirmed stable — a cell with 40 slices and only 2-3 still settling near skin never pays for redoing the other 37. The report lists exactly which slice(s) needed more than one pass and how many, plus whether any slice hit the cap still changing.

Max stability passes — hard per-slice cap so no single slice can loop forever; if it's still changing after the cap, the tool stops and tells you which slice(s), rather than continuing indefinitely.

Copy Label to Adjacent Slice

Copies one label's 2D shape from the currently-viewed slice onto the next or previous slice — useful for patching a slice where a cell's cross-section is missing or broken, by reusing a clean neighboring slice's shape instead of hand-painting it.

Label to copy — type it directly, or click the label in the viewer and use **Use selected**. Accepts any positive label ID or **-1** for skin (0 and any other negative value are rejected).

Copy to — choose **Next slice (Z+1)** or **Previous slice (Z-1)**.

Click **Copy Label to Adjacent Slice**. The label's own old shape on the target slice is cleared first, then the copied shape is painted in — running this again on the same pair of slices replaces the previous

copy cleanly rather than leaving a stale double outline. As with Correct Label, a neighboring label's pixels on the target slice are never touched: any part of the copied shape that would land on a different label is silently dropped, and the status message reports how many pixels were skipped that way so a partial-overlap copy is never mistaken for an exact one.

Correct Adjacent Labels

For two labels that end up touching or merged on **one slice** — two real cells, or a cell and a skin-residue fragment — e.g. right after Copy Label to Adjacent Slice pastes a shape that now touches a neighbor there. **2D only, current slice.**

Why not just run Correct Label twice: at a shared contrast threshold, both labels' regenerated regions can fuse into one connected blob exactly where they touch, and each single-label correction's own foreign-label guard would draw the boundary from the *other* label's stale original shape — not the real signal boundary between them, which is usually different once both are corrected.

How the cut is placed: this tool regenerates **both labels together** from the intensity threshold — every pixel in the working rectangle at or above the threshold, not just what already connects to either label (see the same note under Correct Label) — then splits that combined region with a watershed **seeded by each label's own existing footprint** — not by auto-detecting the two strongest intensity peaks anywhere in the combined region the way Split Label's 2D mode does. That distinction matters: a combined region can have more than one real dip (e.g. genuine internal texture inside one of the two cells, in addition to the real seam between them), and blind peak-finding can lock onto the wrong one entirely. Seeding directly from each label's own current region means the correction floods outward from each cell's own known territory, so the two fronts meet near the *actual* boundary's neighborhood — following the real signal there, not just re-drawing the old shape, but also not wandering off to some unrelated dip elsewhere. Any signal island not directly reachable from either label at all still gets assigned to whichever one is nearer (so nothing in the box is silently dropped), with the same clean 1-pixel gap left at the dividing line either way.

The working rectangle is Label A's own bbox + padding — not the union with Label B. Label A is the focus: this is the correction "for" A, using B only to resolve the local boundary right where the two actually

meet. If B is larger, or its own extent reaches far beyond A's own neighborhood, none of that distant territory is ever read or touched — only whatever portion of B happens to fall inside A's own (padded) rectangle takes part, exactly like any other foreign label would if it happened to be there. If B doesn't reach into A's rectangle at all, the tool refuses with a clear message rather than doing nothing useful (increase padding, or correct A alone with plain Correct Label instead).

Signal layer — pick the Image layer whose current contrast window should be used.

Label A / **Label B** — type directly, or click each label in the viewer and use its own **Use selected** button. Order matters: A is the label the working rectangle is sized from. Either field accepts any positive label ID or **-1** for the Protect Skin as Label sentinel (0 and any other negative value are rejected) — e.g. resolving a real cell's boundary against skin the same joint way as against another cell.

Bbox padding (px) — default 15, around Label A's own existing footprint on the current slice.

Click **Correct Adjacent Labels**. A third, unrelated label is never touched, absorbed, or grown into — same protection as every other Correct Label tool. If the threshold leaves one label's own existing footprint with nothing to anchor to at all, the tool refuses rather than silently erasing that label — adjust the contrast window and try again. The status message reports each label's final pixel count (within the working rectangle — Label B's own territory outside it, if any, is unaffected) and how many pixels (if any) were lost right at the cut boundary.

If Soften label contours (sanding) (Common Settings) is checked — the default — **both** Label A and Label B are sanded immediately afterward, each independently, same foreign-label protection. The status message reports how many of the two were actually softened.

Auto-grow until signal clears the border — same behavior as Correct Label's own auto-grow, just seeded with both Label A and Label B from the start instead of one label — the working rectangle stays scoped to Label A throughout, exactly as above, even as growth retries with a bigger pad or folds in a third label. If growth reveals a third label, it's folded into the joint correction too (only via whatever portion of it falls inside A's own rectangle). Same **Growth step (px)** / **Max growth iterations** fields, same non-convergence reporting.

Convergence is judged on Label A alone. Label B is the adjacent label, usually larger or reaching further than A's own rectangle — it will typically keep touching the edge of that rectangle no matter how much the pad grows, since it was never meant to be grown to its own true extent here. Only Label A's own border status decides whether auto-grow stops; Label B touching the edge is expected and doesn't trigger another retry.

Keep re-running until the boundary stabilizes — default off, **independent of auto-grow above** (usable with it, without it, or instead of it), the same mechanism Correct Label's own 3D-mode stability switch uses, applied here to the single-slice two-label case. The joint watershed split above doesn't just shape Label A — it also redraws Label B's (and any folded-in third label's) boundary at the same time, seeded from each label's CURRENT shape. Feeding one pass's result back in as the next pass's starting point lets the shared boundary settle a little closer to a mutually-consistent position each time, instead of stopping after just the first (possibly still-settling) placement. Re-runs until **every** label in the group has a shape identical to the previous pass, or **Max stability passes** (default 10) is hit — a fresh group still needs a minimum of 2 attempts to *confirm* nothing is changing, exactly like Correct Label's own 3D stability switch.

Protect Skin as Label

Turns everything OUTSIDE the brain mask into a real, ordinary label (ID **-1** by default) — so it's structurally protected by the exact same foreign-label exclusion every Correct Label / Correct Adjacent Labels / auto-grow / Cellpose-SAM auto-correct call already has, instead of relying on those tools' own intensity threshold happening to stay below skin's own signal everywhere. A real cell's own correction bleeding into skin residue — a genuine, recurring failure mode even with a correctly-scoped local working area, since skin can be just as bright as real signal — is exactly what this rules out structurally: skin becomes just another label nothing else is ever allowed to grow into, absorb, or jointly split against.

How it works: one click does both steps. First, every background voxel outside the brain mask is bulk-filled with the skin label (never overwriting a real cell's own territory, never touching anything inside the brain mask). Then that label is immediately run through the same 3D per-slice correction engine Correct Label uses — trimming it down from "the whole outside-brain region" to just the real, signal-supported skin territory (everything at or above the signal layer's own contrast

lower limit). The one difference from a normal 3D correction: skin never jointly re-derives a boundary against a real cell it happens to be touching — it only ever excludes it, exactly like any other foreign label, since skin borders essentially every cell in the fish and a joint watershed split against each one would be both wrong (skin isn't "the same kind of thing" as a cell whose boundary needs resolving) and needlessly expensive.

Never claims anything inside the brain mask, even during the trim. Skin's own working area, once bulk-seeded, already touches the image's own edges on essentially every slice — the generic correction engine's own "real signal, not already claimed by a different label" rule has no way, on its own, to tell real skin residue apart from an unrelated unclaimed signal blob sitting *inside* the brain (debris, or a real cell that hasn't been segmented yet) — both look identical to it: unclaimed background with signal above the threshold. This tool explicitly strips any voxel the trim step would otherwise have claimed if it falls inside the brain mask, restoring skin to background there instead — this can only ever give voxels back, never take any away, so it can't affect the real outside-brain result. The status line reports how many pixels were kept out of skin this way, when any were.

Signal layer — the Image layer whose current contrast window sets the threshold, same convention as every other Correct Label tool.

Brain mask layer — the Labels layer holding the brain mask (populated from any Labels layer in the viewer — typically `<stem>_brain_mask`, already created by a normal Tab 1 run).

Bbox padding (px) — same meaning as elsewhere, though largely irrelevant here since skin's own local working area on any slice is already close to the whole slice.

Click **Protect Skin as Label**. The status line reports the skin label's own final pixel count, and — if any real label ended up touching or sitting near skin anywhere in the fish — **which ones**, so you know exactly which cells sit closest to the brain edge and are worth a closer manual look (they're the ones most at risk of already having a pre-existing bleed from before this run, which protecting skin now doesn't retroactively undo on its own).

Remove Skin Label — clears every voxel currently equal to the given label ID back to background, a plain bulk clear (not connected-component-restricted). The field is pre-filled with whatever ID the last

“Protect Skin as Label” run used; use it whenever the skin label is no longer needed, e.g. right before Save Labels or Statistics if you don’t want it in the final output.

Save Labels

Opens a file-save dialog pre-filled with the output folder (see Section 10) and the current layer name as the filename. Choose a location and filename, then click Save.

Labels are saved as `int32` TIFF. Each voxel value = label number (0 = background).

> **Save Labels is separate from Create Labels by design.** This lets you edit labels in napari (split, delete, merge) before saving the final result.

> After saving labels, switch to **Tab 3 — Statistics** to compute measurements for each cell.

---

7. Tab 3 — Statistics

This tab computes a comprehensive set of morphological, spatial, intensity, and brain-region measurements for every label and saves them to a CSV file. It is intentionally separate from Tab 2 so there is room to configure all options comfortably before clicking Generate.

For what each output column means, see Section 11 below. For the algorithm/formula/library behind each one — useful if you’re auditing a result or citing the method — see the separate STATISTICS_GUIDE.md.

> Make sure a Labels layer is selected in napari before using this tab.

Description backend

Selects the engine used to generate the plain-language `description` column in the CSV.

Option	Internet	Cost	Notes
	No	Free	

Option	Internet	Cost	Notes
Rule-based (offline)			Always available; template-based sentences
Ollama (local, free)	No	Free	Runs a local LLM on your machine
OpenAI API (paid)	Yes	Pay-per-token	GPT-4o-mini recommended for low cost
Claude API (paid)	Yes	Pay-per-token	claude-haiku-4-5 recommended for low cost

See Section 12 for detailed setup instructions for each backend.

Ollama sub-panel (shown when Ollama is selected)

- **Endpoint:** URL where Ollama is running. Default: `http://localhost:11434`. Change this if Ollama runs on a different machine or port.
- **Model:** The Ollama model name to use (e.g. `llama3`, `mistral`, `phi3`). Must be pulled first (`ollama pull llama3`).

API sub-panel (shown for OpenAI or Claude)

- **API Key:** Your secret API key. Shown as dots (password field). **Saved encrypted in your OS's credential store** (not the plugin's plaintext config file) and prefilled next session — see the note under Step 2 below.
- **Model:** The model identifier (e.g. `gpt-4o-mini` for OpenAI, `claude-haiku-4-5-20251001` for Claude).
- **Base URL:** Optional. Leave blank unless you use an OpenAI-compatible proxy or self-hosted endpoint.

Intensity statistics (optional)

Image layer dropdown — select an Image layer from your napari session (or leave as “None” to skip).

When an image layer is selected, three additional columns are computed per label using the raw intensity values inside each cell’s mask:

- `mean_intensity` — average pixel intensity inside the label
- `integrated_intensity` — total sum of all pixel values (proportional to total fluorescent material)
- `intensity_cv` — coefficient of variation (std / mean) — a measure of how uniform the signal is; 0 = perfectly uniform, high values = heterogeneous staining

> Select the microglia channel (usually the green channel, ch1) for biologically meaningful results.

Brain regions (optional)

Assigns each cell to a named anatomical brain region and computes its distance to the nearest region boundary.

Boundary lines dropdown — select a Shapes layer containing one or more `line` shapes, or leave as “None” to skip.

Region names text field — enter the region names separated by commas, listed anterior to posterior. For N boundary lines, provide exactly N+1 names.

Example: If you draw one line separating the optic tectum from the hindbrain, enter:

```
Optic tectum, Hindbrain
```

Fish orientation and axis convention:

In these stacks, the fish lies along the **X axis** with the head pointing toward X = 0 (anterior = small X, posterior = large X). Y runs from 0 to 2048 top-to-bottom. The optic tectum / hindbrain boundary therefore runs roughly **top to bottom along Y**, separating the left part of the image (optic tectum, small X) from the right part (hindbrain, large X).

How to draw region boundaries:

1. In the napari toolbar, click **New shapes layer** (or add via Layers → Add shapes layer).
2. Select the **path** tool in the toolbar (a polyline — click once per vertex, double-click to finish). Use **path** rather than **line** so you can follow the curved anatomy of the optic tectum / hindbrain boundary.
3. Click along the boundary curve **from top to bottom** (Y = 0 toward Y = max), following the anatomical contour. The optic tectum will be on the left of your drawn path (smaller X), the hindbrain on the right (larger X). Double-click on the last point to finish.
4. For multiple regions, draw one path per boundary, each running top to bottom.
5. Select the Shapes layer in the **Boundary lines** dropdown and type your region names.

The boundaries are sorted automatically by mean X position of their vertices (leftmost = most anterior). For each cell, the plugin finds the nearest segment on the boundary curve and uses its orientation to determine which side the cell falls on (left = more anterior region, right = more posterior region).

Two additional columns are added to the CSV:

- `brain_region` — name of the region this cell belongs to
- `region_boundary_dist_um` — distance in μm to the nearest region boundary line

This is verified ground truth (checkbox)

Off by default. The Labels layer being measured could be anything — a raw, uncorrected prediction just as easily as a hand-verified fish — and this tab computes statistics on whatever layer is active regardless. Only tick this when the layer about to be measured has actually been manually corrected/verified as real ground truth.

When ticked, clicking **Generate Statistics** does three things in addition to its normal job:

- This run's **smallest** measured cell (`volume_vox`) is folded into the same never-rising **Min volume** floor the Tab 5 GT sweeps maintain (Common Settings, 6a) — exactly as if you had run one of those sweeps against this fish instead.

- This run's **largest** measured cell is folded into a separate, never-falling **Max volume** — the biggest cell volume ever confirmed real across every GT fish checked so far (Common Settings, 6a). Like Min volume, there is no slider for it — it isn't a pipeline parameter, only the number `is_volume_outlier` (below) is compared against.
- The active Labels layer's own **smallest real hole** is measured directly (the same way the Pixel Classifier's two GT sweeps and the Cellprob/Large-contact sweep already measure it) and folded into the shared, never-rising **Min hole size** floor — this is a real slider (Common Settings, 6a), so a GT-verified Statistics run moves it visibly.

All three give a lighter-weight path to contributing GT evidence beyond running a dedicated Tab 5 sweep, without ever risking an unverified prediction accidentally moving any of them to something wrong. Leave the checkbox unticked for any exploratory or unverified run — the CSV is still produced and still flags outliers against whatever bounds were last measured, it just can't move any of them itself.

The checkbox always un-ticks itself once Generate Statistics finishes, whether or not it was ticked for that run. It's a one-shot "count this specific run as GT" arm, not a sticky mode — a later exploratory run right after a GT-verified one still requires deliberately re-ticking it, so it can never silently ride along and contribute unverified data just because the box happened to still be checked from before.

Generate Statistics (button)

Click to compute. Runs in a background thread. When complete:

- A CSV file is saved to the output folder (see Section 10), named `{source_file_stem}_statistics.csv`.
- The status line shows how many labels were processed, and, if the ground-truth checkbox above was ticked, what this fish's smallest/largest measured cells and smallest real hole were, and what the updated Min volume floor / Max volume / Min hole size floor now are.

The CSV contains one row per label with up to 47 columns depending on which optional features are enabled. See Section 11 for a full description of every column.

Every run — GT-verified or not — includes an `is_volume_outlier` column, `True` for a cell on **either** side of the two GT-tracked bounds:

- **Bigger than the outlier ceiling.** Neither route ever deletes an oversized cell automatically — it's flagged for a human to look at instead, since "unusually large" is far more often a real biological outlier or an under-corrected merge of two touching cells than definite debris.
- **Smaller than the Min volume floor.** Both routes only auto-remove a cell once it drops below `Final min-size fraction × Min volume` (the golden-ratio cutoff, 6a — Cellpose-SAM's final safety-net stage in 6c, the Pixel Classifier's own volume filter in 6b) — anything above that deletion line but still under the confirmed floor itself survives untouched on either route, yet is still smaller than any cell ever proven real. Flagging it here (rather than silently trusting it) surfaces exactly that gray zone for review.

Score Against GT — moved to Section 9e, Tab 5 — Sweeps & Utilities. Scores any two Labels layers already in the viewer against each other.

---

8. Tab 4 — AI Tools

Always visible, regardless of GPU. A banner at the top reports your GPU situation and adjusts its tone accordingly — checked once when napari starts:

Situation	Banner
No CUDA GPU detected	Red, bold: training/ inference will run on CPU — can take **days to months** for a full run instead of hours. Still usable for small experiments.
CUDA GPU present, under 8GB VRAM	Amber, bold: training may still work with a **reduced `batch_size`** (try 2, or even 1) but could be slow or hit out-of-memory errors.
CUDA GPU present, ≥8GB VRAM	Green, quiet confirmation — no action needed.

This used to be a hard gate — the whole tab was hidden below 8GB VRAM. Changed deliberately: a smaller GPU, or none at all, doesn't mean the tools are useless, just slower, or in need of a smaller `batch_size`. GT Annotation itself has never needed a GPU either way.

Email notification (optional) — moved to Section 9h, Tab 5 — Sweeps & Utilities, General category. Each training launcher below has its own "Email me when this training run stops" checkbox that opts into it.

A switch below that picks one of two mutually-exclusive groups. Only one is shown at a time.

8a. GT Annotation

Hand-draw polygon boundaries on key slices to create brain/skin ground-truth masks — the manual annotation step that produces training data for the MONAI model.

1. **Image layer** — pick the Image layer to annotate from the dropdown (populated from whatever's open in the viewer — use **Open TIF / IMS file** in Tab 1 first if nothing is loaded yet). Selecting a layer here automatically creates a `brain_polygons` Shapes layer (yellow) if one doesn't already exist.
2. **Draw polygons** — select the `brain_polygons` layer in the Layers panel, choose napari's polygon tool, and trace the brain boundary on key slices — roughly every 10 slices (e.g. 0, 10, 20, 30…). You don't need to draw every slice: the polygon on slice 90 is automatically propagated to all slices beyond it.
3. **1. Interpolate Polygons** — smooths and resamples each drawn polygon to 96 points, then interpolates point-to-point between key slices along Z. Produces a `brain_polygons_interpolated` layer (cyan) — review it before continuing.
4. **2. Generate Masks** — rasterizes the interpolated polygons into a brain mask, saves `brain_mask.tif`, `skin_mask.tif`, `original.tif`, `brain_only.tif`, `skin_only.tif`, and both polygon `.npz` files to `<source_folder>/<source_stem>/` (the same output-folder convention as every other tab — see Section 10). Also adds `brain_mask`, `skin_mask`, and a new `brain_only` layer to the viewer, without touching or hiding any of your other layers.

> **Note:** unlike Tabs 1-3, this section doesn't have its own file-open button — it always annotates whichever file was most recently opened via **Open TIF / IMS file** in Tab 1, matching that same output-folder convention.

8b. MONAI Training

Prepares training data and launches MONAI U-Net training — the model Tab 1 uses for skin removal.

Prepare Training Data converts raw+GT fish folders (the output layout GT Annotation produces) into the HDF5 dataset the trainer needs. Leave the brain/skin directory fields blank to use the training script's own built-in defaults, or list your own comma-separated paths. Takes a few minutes; runs in the background without blocking the UI. On success, auto-fills the Train MONAI section's data directory.

Train MONAI U-Net launches the actual training run — configure `epochs`/`batch_size`/`lr`/`val_every`/`ckpt_every`/GPU index, optionally point `resume` at an existing checkpoint to continue training instead of starting fresh, then click **Launch Training**. See How training launches work below for the shared **Patience (checkpoints)** early-stopping field and what happens next.

8c. Cellpose-SAM Training

Extracts fine-tuning crops and launches Cellpose-SAM training — the model Tab 2's Cellpose-SAM Segmentation uses.

Extract XZYZ Patches generates 2D training crops in all three orientations from a full-fish image + GT labels pair: **XY** at native resolution, **XZ**/**YZ** with the Z axis stretched by the voxel-scale-derived anisotropy so all three orientations end up at the same effective pixel scale. Only crops containing GT signal are kept, up to `max/orientation` per orientation. This is the method behind every real Cellpose-SAM training dataset in this project (`train_cellpose_512`, `_multi`, `_multi3`).

> An earlier bbox-crop extraction tool (single/double/triple/quadruple crops per cell) used to sit here too. Removed 2026-08-09 — it reproduced the same approach that produced this project's first real fine-tuning attempt (April 2026), which was worse than the untrained base model on every validation

> patch and was abandoned at the time. The code is preserved in `skin_segmentation/crop_extraction_plugin_port.py` for reference, not as a usable tool.

- **crop_size** / **crops/slice** / **max/orientation** / **min_gt_pixels** / **seed** — control crop dimensions and how many are sampled; defaults (512 / 5 / 320 / 10 / 42) match what's been used for every real dataset so far.
- **Clean truncated labels after generation** (checked by default) — a crop framed around one target cell can also graze the corner of a *different* nearby cell purely by chance, sometimes showing only a tiny sliver of it. Left in, that sliver is still a valid-looking label with a wildly wrong flow-center target (Cellpose points flow vectors toward each object's own centroid — a fragment's visible centroid is nowhere near the real cell's center). With this on, any label whose crop-visible pixel count falls below **Minimum visible fraction to keep a label** (default 90%) of its true full-slice cross-section gets zeroed out of that crop — automatically, right after generation, not as a separate step you have to remember. A crop's own intended target cell is essentially never affected (it's already well above this threshold in the crop it was generated for); this specifically catches incidental neighbors. Before writing anything, the whole crop folder is backed up to `<folder>_pretrunc_backup` — skipped on a second run if that backup already exists, so re-running never overwrites an earlier backup with already-cleaned files.
- This is the exact fix already applied by hand to this project's real training data (D1F1/D1F2/D1F4, 2026-08-05) — now the default behavior for every future extraction, not a one-off research script.

Train Cellpose-SAM launches fine-tuning — configure `n_epochs`/`batch_size`/`save_every`/`log_every`/`lr`, then click **Launch Training**. The `pretrained` field defaults to whatever checkpoint is already loaded in Tab 2's Cellpose-SAM Segmentation section — i.e. by default this **continues training from where Tab 2 left off**, though you can browse to a different starting checkpoint (or type a builtin name like `cpsam`) if you want to start fresh. `branch_weight`/`branch_radius` control the project's branch-weighted loss (weights thin/branch-tip pixels more heavily during training so the model doesn't under-segment fine processes) — set `branch_weight` to `0` to disable it and use the standard Cellpose loss instead.

Calibrate branch_radius (from GT) measures the real branch thickness of actual GT-labeled cells instead of guessing `branch_radius` by hand. Browse to a GT labels volume, set **scale Z**/**scale XY** to match

its voxel scale, tick **“This is verified ground truth”** (off by default, same one-shot rule as every other sweep tool — see Section 9’s intro), and click **Calibrate branch_radius**. The tool 3D-skeletonizes every labeled cell, decomposes each skeleton into branch segments, measures each segment’s mean diameter via an anisotropic distance transform, and takes the **thinnest quartile** (the distal branch tips — the fine processes `branch_weight` exists to protect, as opposed to thick soma-adjacent segments) as the basis for the recommendation, converting that radius from microns to pixels at the given scale. If GT-verified, the result feeds a never-falling ceiling tracked across every fish calibrated so far (a thicker “thin branch” measured in any fish sets a higher bar the field must still meet, the opposite direction from Min volume’s never-rising floor) — that ceiling, not just this run’s own measurement, is applied to the `branch_radius` field above (and shown in a read-only **Recommended branch_radius** line underneath it) and saved to config, no manual copy-over. Left unticked, the run still reports what it measured but changes nothing. This can take anywhere from several seconds to a couple of minutes depending on how many cells are in the GT volume; it runs in a background thread so napari stays responsive.

Verify Best Epoch (GT Sweep) — moved to Section 9d, Tab 5 — Sweeps & Utilities. Confirms or corrects the recommended-checkpoint pointer above against real GT.

How training launches work

Both “Launch Training” buttons (MONAI and Cellpose-SAM) start a **detached background process** rather than running inside napari itself — the command runs via `conda run -n <env> --no-capture-output <script> ...`, launched so it keeps running even if you close napari (technically: `setsid()` on Linux/Mac, `CREATE_NEW_PROCESS_GROUP | DETACHED_PROCESS` on Windows). This works identically on all three platforms without needing `tmux` (which isn’t available on Windows at all).

- **Live status** — a log-tail view in the GUI refreshes every 8 seconds (deliberately coarse — there’s no benefit to checking more often on an hours-to-days job).
- **Patience (checkpoints)** — an integer field in both groups, and it’s the *same rule* for both: stop automatically once N checkpoints in a row pass with no improvement in the model-selection metric (Full-brain Dice for MONAI — higher is better; `test_loss` for Cellpose-SAM — lower is better; the plugin handles the direction per metric

automatically). `0` disables early stopping entirely. This is enforced by the plugin itself, reading each checkpoint as it lands in the log — `train.py`'s own built-in `--patience` flag is always overridden to an effectively-infinite value so it can't quietly stop training before the GUI's check does; there's exactly one early-stopping mechanism, not two that happen to look the same in the UI.

- **Reopening napari mid-training** — the plugin remembers the running job (including the patience setting) and automatically reconnects to it, so you don't lose visibility into a training run just because you closed and reopened napari. You'll see "Resumed monitoring PID …" instead of an empty status.
- **Reopening napari *after* the job already finished** — if the training process is no longer running by the time you reopen napari (e.g. it ran to completion, or crashed, while napari was closed), the status line reports that immediately: "Training (PID …) finished while napari was closed. Best … at epoch …". For Cellpose-SAM this is also the point where the recommended-checkpoint pointer (see below) gets written, if it wasn't already. Either way you don't need to have napari open at the exact moment a run finishes — the next time you open it, it tells you what happened.
- **Email notification** — see Section 9h, Tab 5. Unlike the previous two bullets, this one doesn't depend on ever reopening napari at all — the email arrives on its own schedule regardless.
- **Stop Training** — kills the training process and everything it spawned (the `conda run` wrapper spawns a child `python` process, and both are terminated together). Early stopping uses this same kill mechanism internally.
- **Which checkpoint to use afterwards** — MONAI's `train.py` already tracks and saves its own best checkpoint as `best_model_fullstack.pth`, so nothing extra is needed there. `train_xzyz.py` (Cellpose-SAM) has no such tracking — it only saves periodic epoch checkpoints — so whenever the plugin observes a Cellpose-SAM run has stopped (finished on its own, early-stopped, or discovered already-finished the next time you reopen napari — see above), it writes a small pointer file, `<model_name>_best_recommended.txt`, into the run's `models/` folder next to the checkpoints. It's a one-line text file naming the best-scoring checkpoint (by `test_loss`), e.g. `cpsam_microglia_xzyz_epoch_0150` — not a copy of the (often 100s-of-MB) checkpoint itself, and not an OS symlink either (those need elevated privileges/Developer Mode on Windows), so it works the same way on every platform with no special permissions. The GUI's status line also reports the best epoch directly once the run stops.

- **If a script isn't found** — `prepare_data.py`/`train.py`/`train_xzyz.py` ship with the plugin (bundled under `napari_zf_microglia_ai/training_scripts/`, installed as package data), so this shouldn't normally happen. If you want to point at a locally modified copy instead, override the path via the `monai_prepare_script_path`/`monai_train_script_path`/`cellpose_train_script_path` keys in `~/.config/napari-zf-microglia-ai/config.json`.

---

9. Tab 5 — Sweeps & Utilities

Seven tools, consolidated here from Tabs 1-4 (where they used to sit right alongside — and clutter — the primary pipeline controls). Each is individually collapsible: click a section's title checkbox to hide its contents, so you can keep only the one you're actively using expanded. Every tool below still operates on its *original* tab's own sliders/fields and auto-applies its findings back there — moving where a tool is displayed doesn't change what it reads from or writes to. Five are GT-*sweep* tools (test a small parameter grid against a handful of proxy cells or one mask, as a fast approximation); the other two (Score Against GT, Build GT-Correction Package) are related GT utilities that don't fit that "sweep" shape but belonged with the others more than with their old tab's core workflow.

"Show tools for…" filter — with seven tools stacked in one tab and no indication of which pipeline each belongs to, it wasn't obvious at a glance what any given tool was even for. Four checkboxes at the top of the tab let you hide the ones you don't need:

Category	Tools shown
Skin Removal (MONAI)	9a. Verify MONAI Threshold / Erosion
Pixel Classifier segmentation	9b. Verify BG Threshold / Erosion, 9g. Verify Smooth σ XY / σ Z
Cellpose-SAM segmentation	9c. Verify Cellprob / Large-contact, 9d. Verify Best Epoch, 9f. Build GT-Correction Package
General (any pipeline)	9e. Score Against GT

All four are checked by default (nothing is hidden until you actually uncheck something), and your choice is saved to config and restored next time you open napari. Unlike Tab 4's MONAI/Cellpose-SAM

training switch, these are independent checkboxes, not a mutually-exclusive radio choice — you can leave several checked at once if you work with more than one pipeline.

Every sweep here keeps a running history across every fish you've ever swept, not just the one you swept last. A single sweep run only ever proves something about the one fish it ran against; treating that one result as the final answer, and overwriting whatever an earlier sweep on a different fish had found, throws away real evidence for no reason. Each tunable value therefore falls into one of two categories, and the tool remembers a per-fish history for whichever one applies:

- **Floors and ceilings** (Min volume, Min hole size, branch_radius) protect against a specific failure — discarding a real cell, erasing a real hole, or under-protecting a real thin branch. These track the single most demanding piece of evidence seen across every fish: Min volume/Min hole size only ever get *stricter* (lower) as a new fish proves an even smaller real example exists; branch_radius only ever gets *more generous* (higher) as a new fish proves an even thicker "thin" structure needs covering. A value already proven safe by one fish is never silently relaxed by a later sweep that simply didn't happen to test anything as extreme.
- **Everything else** (BG Threshold, Erosion, Smooth σ XY/Z, Cellprob, Large-contact, MONAI Threshold) has no safety direction — each fish's sweep just finds that one fish's own local optimum, and there's no reason to trust the most recent fish over any earlier one. These are **averaged** across every fish swept so far instead.

Re-running a sweep against a fish you've already swept before, for example after correcting that fish's ground truth, updates that fish's own entry in place rather than adding a duplicate — the history is keyed by GT filename, so it stays accurate rather than growing stale entries forever. Every sweep's status message reports both numbers now: what this specific fish's sweep found, and what the aggregated recommendation is once that fish's result is folded in.

"This is verified ground truth" — every sweep tool below now has one, exactly like Tab 3 Statistics. Off by default. A sweep still runs and reports its own result (best point, score) regardless of whether it's ticked — what it *doesn't* do without it is move any of the shared values above, apply anything to a live slider, or (for Verify Best Epoch) rewrite the active checkpoint. The GT labels path each sweep takes could be anything — a raw, uncorrected prediction just as easily as a hand-verified fish — so only tick it once you're sure that path really is verified ground truth. **The checkbox always un-ticks itself once the**

sweep finishes, GT-verified or not — a one-shot arm per run, not a sticky mode, so a later sweep right after a GT-verified one still needs deliberately re-ticking it.

Sliders/fields that a sweep can auto-apply a value to now show a separate, read-only "Recommended: X" line underneath them, in whichever tab that control actually lives (Tab 1 for MONAI Threshold/Erosion/BG Threshold, Tab 2 for Smooth σ XY/σ Z/Cellprob threshold/Large-contact merge, Tab 4 for branch_radius) — distinct from the slider itself, which stays freely editable at all times. A GT-confirmed sweep both applies its recommendation to the live slider *and* updates this read-only line; moving the slider afterward to test something else never touches the recommended line, so what the evidence actually supports is never silently lost. Min volume/Max volume (Common Settings, 6a) already work this way by design — informative value only, no slider at all — since nothing should ever hand-tune them in the first place.

9a. Verify MONAI Threshold / Erosion (GT Sweep)

The cheapest of the four sweepers here. Checks MONAI's own brain segmentation — is the current **MONAI Threshold** and **Erosion** (both Tab 1 fields) combination actually the one closest to a hand-corrected brain mask? Unlike the other sweepers, this scores a single whole-volume mask, not multiple per-cell labels — Dice/IoU/precision/recall between the predicted brain mask and a GT mask directly, no complex-cell selection or bounding-box cropping needed.

1. **Image** — the raw volume to run inference on. Must be a **TIFF, not `.ims`** (loaded via `tifffile.imread`, unlike Tab 1 itself which does support `.ims`), and must be the true pre-MONAI raw image — feeding it an already brain-masked image would bias the very segmentation this tool is scoring.
2. **GT brain mask** — a *hand-corrected* brain_mask.tif, e.g. from **GT Annotation** in Tab 4 (the polygon annotation tool's own rasterized output) — not a MONAI prediction.
3. **Threshold min/max/step** and **Erosion min/max/step** — define the grid. Defaults (0.15–0.35 step 0.05, 0–4 step 1) span 5×5=25 points centered on the recommended threshold.
4. Click **Run Threshold/Erosion Sweep**.

MONAI's sliding-window inference (the only genuinely expensive, GPU-bound step) runs **exactly once**, producing a raw probability map. Every threshold and erosion value in the grid is then just a cheap re-threshold + largest-component/fill-holes + optional erosion on that

same probability map — no reloading the model, no repeat sliding-window passes. A full 25-point grid typically finishes in well under a minute on GPU, and still works (just slower) on CPU/MPS since it uses the same device selection as **Run Skin-Remover**.

The report is a 2D grid (rows = Erosion, columns = Threshold, cells = Dice%), with your current Tab 1 slider values marked. Once the sweep finishes, this fish's best point is folded into a running average across every fish swept so far (Erosion's history is shared with the BG Threshold/Erosion sweep below, since both tune the same Tab 1 slider), and **that average is applied to the Tab 1 Threshold/Erosion sliders and saved to config** — no manual copy-over needed, and the recalibrated values persist across napari restarts.

9b. Verify BG Threshold / Erosion (GT Sweep)

Answers the same kind of question as 9d below, but for the Pixel Classifier path instead of Cellpose-SAM: is the current **BG Threshold** (Tab 1) and **Erosion** (Tab 1) combination actually the one that produces microglia labels closest to ground truth, or does a nearby combination do better?

1. **GT image** — the full-fish raw/brain_only image, same one Tab 1 ran on.
2. **brain_mask.tif** — the *raw* (un-eroded) mask Tab 1 saves. MONAI inference itself is **not** re-run by this sweep — it only varies what happens *after* inference (erosion, background thresholding, labelling), so it needs an already-computed mask from a normal Tab 1 run rather than the model checkpoint.
3. **GT labels** — the corrected ground-truth microglia label volume for that fish.
4. **BG Threshold min/max/step** and **Erosion min/max/step** — define the grid. Defaults (1.0–1.8 step 0.2, 0–4 step 1) span 5×5=25 points centered loosely on the recommended BG Threshold.
5. Click **Run BG/Erosion Sweep**. For each grid point, it: finds the N most complex GT cells (same branch-count ranking as the Cellpose-SAM sweep, computed once), applies that erosion + BG Threshold to each cell's cropped region, runs Create Labels (using this section's own σ XY / σ Z above, plus **Min volume** and **Min hole size**, both measured automatically from the GT itself — see below) on the crop, and best-IoU-matches the result against GT.

Min volume and Min hole size are both measured from the GT, not read from their sliders. The small-blob cleanup threshold used during the sweep is the true smallest labeled cell's own voxel volume in the GT labels you provided, not whatever the Min volume slider (Tab 2) happens to show — a fixed guessed number (this used to default to a flat 7500 regardless of fish) risks discarding a real small cell as noise if it's too high. Min hole size works the same way in reverse: it's the smallest genuinely real internal gap found anywhere in that GT's own cells (scanning each cell's per-slice footprint for background regions a human annotator deliberately left unlabeled), so the sweep never fills in a gap the ground truth itself confirms is real. Single-pixel gaps are treated as annotation noise rather than evidence when measuring this, since real GT checked during development showed those are common and unrelated to genuine structure.

A small text line below Tab 2's Min volume slider, and another below Min hole size — **"Recommended minimum/floor (from GT sweeps so far): N vox"** — tracks the running floor across every sweep you've run, independently of the slider itself. This tracking is deliberately *not* the same thing as "whatever the slider currently shows": both sliders stay fully user-editable like every other field in this plugin (e.g. to test a different value by hand), but that manual experimentation should never corrupt the evidence-based recommendation. So each recommendation only ever **decreases** — once one fish's GT proves a cell (or a hole) of a given size is real, a *different* fish's sweep (which may simply lack any cell or hole that small) can't raise it back above that. Each sweep still auto-applies both recommendations to the live sliders as a convenient default — but feel free to change either slider afterward for your own testing; the recommended-value text lines keep the real numbers safe regardless.

This sweep is considerably cheaper than the Cellpose-SAM one: MONAI inference only ever runs once (outside this tool, via a normal Tab 1 run), and neither Erosion nor BG Threshold require reloading a model. A full 25-point grid typically finishes in minutes, and works on CPU too (Create Labels already has a CPU fallback).

The report is a 2D grid (rows = Erosion, columns = BG Threshold, cells = average IoU%), with your current Tab 1 slider values marked and compared against whatever the sweep found best. Once the sweep finishes, this fish's best BG Threshold/Erosion point is folded into a running average across every fish swept so far (Erosion's history is shared with the MONAI Threshold/Erosion sweep above), and Min volume/Min hole size are updated as never-rising floors — **the**

resulting averages and floors are applied to the Tab 1 BG Threshold/Erosion sliders, Tab 2's Min volume and Min hole size sliders, and saved to config.

> This tool depends on Erosion and BG Threshold actually composing correctly in Tab 1's own pipeline. An earlier version of `_on_run` silently discarded Erosion whenever any Background mode was active (the final mask always used the raw, un-eroded mask in that code path) — fixed as of this version.

9c. Verify Cellprob / Large-contact (GT Sweep)

Sweeps **Cellprob** × **Large-contact merge** (both Tab 2, Cellpose-SAM Segmentation) against a full-fish GT labels volume, scored with the exact same whole-fish Hungarian-matched methodology as **Score Against GT** (9e below) — this is how the current defaults were actually found historically (e.g. the cellprob=-2.5/large_contact=20 combination), now automated instead of requiring a CLI sweep script.

1. **Image** / **GT labels** — a full-fish `brain_only` image + its corresponding GT labels volume.
2. **Voxel scale Z/XY** — drives the do_3D `anisotropy` parameter (Z/XY ratio); independent of whatever's open in the viewer.
3. **Cellprob min/max/step** and **Large-contact min/max/step** — define the grid.
4. Tick **"Email me when done"** if you want a notification (~3h is well past the point where that's worth it) — see Section 9h.
5. Tick **"This is verified ground truth"** if the GT labels above are genuinely hand-verified (off by default — see Section 9's intro for the one-shot rule shared by every sweep tool here). Only then will this run's findings move Cellprob threshold, Large-contact merge, the Min volume floor, or the Min hole size floor.
6. Click **Run Cellprob/LC Sweep**. Uses Tab 2's current **Safe-merge max gap** and **Safe-merge min contact** values — only Cellprob and Large-contact vary.

Cellprob is now cheap to sweep, not just Large-contact. Cellpose's own `CellposeModel.eval()` internally splits into two independent steps: the network forward pass that predicts a flow field (the one genuinely expensive, GPU-bound part — completely unrelated to Cellprob or any other threshold) and a separate, cheap mask-formation step that Cellprob threshold feeds into. This sweep now runs the network pass **exactly once** for the whole grid, then re-thresholds cheaply for every Cellprob value, then runs GMM cleanup + Krendl safe-merge per

Cellprob value, with **Large-contact** varying freely on top of that as before. Total sweep time is now roughly **one `do_3D` network pass, period** — not one per Cellprob value.

Flow is not swept, and has no user control anywhere in this plugin: reading `cellpose/dynamics.py` shows its flow-error QC filter only runs when `do_3D=False` (2D/stitch mode) — under `do_3D=True`, which this plugin always uses, changing Flow threshold changes nothing about the result. It's fixed internally purely because `do_3D`'s call signature still accepts it — see Flow threshold in Section 6c.

This used to be by far the slowest of the four GT-sweep tools, because it ran on the full fish rather than a handful of cropped cells — that's no longer true. A single `do_3D` network pass on a full-size fish has historically taken around 3 hours in this project (e.g. D1F4: ~187 minutes), and that's now the sweep's entire cost, regardless of how many Cellprob or Large-contact values are in the grid. **Stop Sweep** only cancels between grid points, and since the network pass now happens once upfront it can't itself be interrupted mid-pass — but that pass is also the whole sweep's cost now, not a multiplier on it. This does **not** run detached — it won't survive closing napari. The report box streams Cellpose's internal `do_3D` progress live during that one pass rather than sitting on one static message — see the note under Run Cellpose-SAM Segmentation in Section 6c.

The shared Min volume field is also recalibrated every time you run this sweep — measured directly from the GT labels volume's own smallest labeled cell, rather than a frozen historical constant. This used to be tracked as a separate "GT-min" value with no never-rising-floor protection at all (a bug in an earlier version of this tool: it just overwrote GT-min with whatever the latest sweep measured, fixed alongside unifying it with Min volume). Cellprob and Large-contact, which have no safe direction to bias toward, are instead averaged across every fish swept so far. This fish's own best point, the updated cross-fish averages, **and** the Min volume floor are all applied to the Tab 2 sliders and saved to config.

Min hole size is measured from GT here too — this route used to just pass through whatever the Min hole size slider already held, never actually looking at this GT's own real holes the way the Pixel Classifier's two GT sweeps (9b, below) already did. It now measures a recommended floor from this GT's own real holes exactly the same

way, via `_pixel_sweep.min_hole_size_from_gt()`, and folds it into the same never-rising `min_hole_size_vox` history every other sweep tool contributes to.

9d. Verify Best Epoch (GT Sweep)

Answers a specific question the recommended Cellpose-SAM checkpoint alone can't: `test_loss` (what picks the recommendation, in Tab 4's Train Cellpose-SAM section) is a proxy for segmentation quality, not the real thing, and checkpoints often plateau within noise of each other. This tool checks the recommendation against actual ground truth on a small, deliberately hard sample:

1. **GT image** / **GT labels** — browse to a full-fish raw/brain_only image and its corrected ground-truth label volume (the same pair used to build training crops). Doesn't need to be a fish the current model was trained on, but usually is.
2. **Recommended epoch** — click **From pointer file** to pull it from `<model_name>_best_recommended.txt` (uses Tab 4's Train Cellpose-SAM section's own Data dir/model_name), or type it in manually if no pointer exists yet.
3. Click **Run Epoch Sweep**. The tool:
   - Finds the **N most morphologically complex cells** in the GT volume (default 5) — ranked by skeleton branch count (most-branched first), sphericity as a tiebreak, *not* by cell size.
   - Crops each to its bounding box + padding (default 15 vox Z, 40 vox XY).
   - Runs `do_3D` inference at the recommended epoch plus **N checkpoints below and above it** (default 2 and 2 — a 5-epoch × 5-cell = 25-inference sweep by default), best-IoU-matches each prediction against its GT cell, and averages.
   - Reports a table plus a plain confirm/disagree verdict against the recommended epoch.

This can take a while — each `do_3D` call is a few minutes, so a default 5×5 sweep is roughly 30 minutes to a couple of hours. **Stop Sweep** cancels between checkpoints (not mid-inference). Unlike Launch Training, this does **not** run as a detached process and does **not** survive closing napari. Tick **"Email me when done"** if you'd rather not watch — see Section 9h.

If the sweep disagrees with the recommendation, applying it — rewriting the recommended-checkpoint pointer to the sweep-confirmed epoch and loading that checkpoint as Tab 2's active

Cellpose-SAM model — now requires **“This is verified ground truth”** (off by default, same one-shot rule as every other sweep tool here) to be ticked for that run. Left unticked, a disagreement is still reported in full, it just doesn’t swap the active model out from under you on the strength of GT labels you haven’t actually confirmed are correct.

9e. Score Against GT

Whole-fish instance-segmentation scoring: Hungarian-matched TP/FP/FN/Score plus mean IoU/Dice (over matched pairs only), between any two Labels layers already loaded in the viewer. This is the same methodology (`compare_pred_gt.py`) this project has used to validate essentially every real modeling decision — checkpoint picks, cellprob/large_contact tuning, before/after model comparisons — ported into the plugin instead of staying a CLI-only script.

1. **Predicted labels** / **GT labels** — pick any two Labels layers from the dropdowns (same shape required).
2. **IoU threshold for a match** — the minimum IoU for a predicted object to count as a true positive for a given GT object (default 0.5).
3. Click **Score Against GT**.

Runs synchronously (pure CPU, `scipy.optimize.linear_sum_assignment`) — fast enough at typical whole-fish object counts that a background thread isn’t needed. **Score** = **TP − 0.5×(FP + FN)**. The report lists every matched pair (IoU%, Dice%, voxel counts, size delta), plus the FN (missed GT) and FP (spurious predicted) object IDs.

This is a genuinely different tool from the four sweepers above: those each test a handful of parameter combinations against a handful of complex cells (or one mask) as a fast proxy; this scores one specific pair of label volumes completely, the way a final reported result would be scored.

9f. Build GT-Correction Package

Packages a Cellpose-SAM correction result for external manual correction — the exact file layout this project has assembled by hand for every fish sent out for ground-truth creation. The corrected result becomes future training/GT data, closing the loop between inference and the training tools in Tab 4.

1. **Fish stem** — the identifier used to name every file in the package (e.g. `NT39-3dpf-D1F4_2024-09-05_15.38.01`).
2. **Source image** — the `brain_only` image the segmentation ran on. **Browsing here also auto-fills every field below** from that fish's own folder (Fish stem, Corrected masks, Raw masks, Output folder) using the established `<parent>/<stem>/<stem>_<artifact>.tif` convention — override anything by hand afterward if it picked the wrong file.
3. **Corrected masks** — the most-advanced Cellpose-SAM correction stage this fish actually has: **sanded** > **auto-corrected** > **Krendl-only**, whichever exists (auto-picked when Source image is browsed above; see 6c's Auto-correct/Sanding sections for what each stage means). Becomes `<stem>_cp_corrected.tif` — the file the reviewer edits first ("start here" per the guide).
4. **Raw Cellpose masks** (optional) — the pre-merge `do_3D` output, if you have it, included as `<stem>_cp_masks_3D.tif` for reference only (not corrected).
5. **Brain mask** (optional) — this fish's own raw (un-eroded) `brain_mask.tif`, if you have it, included as `<stem>_brain_mask.tif` — auto-filled the same way as Raw Cellpose masks. **Without it, whoever corrects this package can't use Protect Skin as Label at all** (that tool needs a brain mask layer), so it's worth including whenever one exists for this fish.
6. **Creation guide** (optional override) — defaults to this project's own `GROUND_TRUTH_CREATION_GUIDE.md`; only set this if it lives somewhere else on your machine.
7. **Output folder** — where the package folder and `.zip` are created.

Click **Build GT-Correction Package**. Output:

```
<output folder>/
├── <stem>_GT_package/
│   ├── GROUND_TRUTH_CREATION_GUIDE.md
│   ├── <stem>_cp_corrected.tif
│   ├── <stem>_cp_masks_3D.tif        (only if provided)
│   ├── <stem>_brain_mask.tif         (only if provided)
│   ├── <stem>_cell_statistics.csv    (label/volume/centroid/bbox
```

```
— quick reference, not the full Tab 3 output)
│   └── <stem>_brain_only_ExtRm.tif
└── <stem>_GT_package.zip            (the folder above, zipped)
```

The statistics CSV is deliberately minimal (label, volume, centroid, bounding box) — a quick reference for someone correcting labels, not the full ~51-column Tab 3 Statistics output.

> **On-disk naming for every Cellpose-SAM stage** is cumulative and self-documenting — each stage's filename is the previous one with one more suffix appended, so reading it left to right tells you exactly which processing steps ran:
>
> | File | Stage |
> |---|---|
> | `<stem>_cp.tif` | raw `do_3D` output, pre-merge |
> | `<stem>_cp_krendl.tif` | + Krendl safe-merge + large-contact merge (always saved by a normal run) |
> | `<stem>_cp_krendl_ac.tif` | + auto-correct (only if that stage was left enabled) |
> | `<stem>_cp_krendl_ac_snd.tif` | + sanding (only if that stage was left enabled too) |

9g. Verify Smooth σ XY / σ Z (GT Sweep)

Checks a parameter every other GT-sweep tool in this plugin had already covered except this one: the Pixel Classifier's pre-threshold Gaussian smoothing (**Smooth σ XY** / **Smooth σ Z**, Tab 2). These have defaulted to 1.5/3.0 since Tab 2 was first built, but — unlike BG Threshold, Erosion, Cellprob, Large-contact, and Min volume, all of which now have a dedicated sweep — they had never actually been verified against real ground truth.

1. **GT image** / **brain_mask.tif** / **GT labels** — same three inputs as 9b above (the raw/brain_only image Tab 1 ran on, the raw un-eroded mask, and the corrected GT label volume).
2. **sigma XY min/max/step** and **sigma Z min/max/step** — define the grid.
3. Click **Run Sigma Sweep**. **BG Threshold and Erosion are held fixed** at whatever Tab 1's sliders currently show — this sweep isolates sigma specifically, the same way 9c holds Flow/Safe-merge fixed while varying only Cellprob/Large-contact.

Cheaper per grid point than the BG Threshold/Erosion sweep: since BG Threshold and Erosion don't change here, each cell's thresholded `brain_only` crop is computed once and reused across every sigma combination — only the `create_labels()` call itself (the smoothing + union-find step) varies per grid point.

Same auto-apply and floor-recalibration behavior as 9b: this fish's best (sigma XY, sigma Z) point is folded into a running average across every fish swept so far and that average is applied to Tab 2's Smooth σ XY/Z sliders, and both Min volume and Min hole size are recalibrated as the same never-rising floors described there.

9h. Email notification (optional)

Not a sweep — a General-category utility, alongside Score Against GT, since it isn't tied to one pipeline. This panel configures **one shared set of credentials**, used by an **"Email me when done"** checkbox next to every long-running tool in the plugin — configure it once here, then opt in per tool wherever you actually want a notification:

- Tab 1 — **Run Skin-Remover**
- Tab 2 — **Run Cellpose-SAM Segmentation**
- Tab 4 — **Launch Training** (MONAI and Cellpose-SAM, each has its own checkbox: "Email me when this training run stops")
- Tab 5 — **Verify Cellprob / Large-contact (GT Sweep)** and **Verify Best Epoch (GT Sweep)**

These are the plugin's operations that can realistically run 30+ minutes; the other, faster sweep/utility tools don't have this option since there's rarely anything to wait for.

Fields: Notify email, **SMTP server/port**, **SMTP username**, **SMTP password**. Leave **Notify email** blank to disable the feature entirely regardless of which checkboxes are ticked elsewhere — that's the default. Free with any Gmail account, no other signup needed. **See Section 12a for the full step-by-step setup** (turning on 2-Step Verification, generating a Google App Password, and what to type into each field) — the short version: server `smtp.gmail.com`, port `465`, username = your Gmail address, password = a Google App Password, not your normal Gmail password, which won't work here.

All four fields are saved between napari sessions — set this up once and every "Email me when done" checkbox works without re-entering anything. The **password specifically is saved encrypted in your OS's**

credential store (Windows Credential Manager / macOS Keychain / Linux Secret Service, via the `keyring` package) rather than the plugin's own plaintext `config.json` — real encryption at rest with an OS-managed key, not something the plugin manages itself. This is reasonable to persist at all because a Gmail App Password is a separate, revocable credential Google issues specifically for unattended third-party use like this, not your real account password. If **Notify email** is filled in but username/password is missing, any tool with its checkbox ticked refuses to start until you either fill both in or untick that tool's checkbox.

On Linux, the OS credential store needs a running, *unlocked* Secret Service session (GNOME Keyring or KWallet) — present on a normal desktop login, but not guaranteed over SSH, on a headless machine, or before you've logged into the desktop once. This is common enough on Linux workstations used mainly over SSH/tmux (confirmed directly on this project's own machine) that the plugin has a second layer for it: if the OS store isn't reachable, the password is saved instead to a local file encrypted with Fernet/AES (via `cryptography`), keyed by a machine-local key file the plugin generates once — no password prompt, no plaintext on disk, works the same way every session. This is a **weaker guarantee than the OS store**, worth being honest about: the key protecting that file lives right next to it, on the same machine and account, so it defends against casual exposure (an accidental `cat config.json`-style leak, a stray backup of just one file, a screen-share) rather than someone with full read access to your home directory. You'll see a one-line note printed to the console the first time this fallback kicks in ("OS credential store unavailable – saved to a local encrypted file instead"). Only if *both* the OS store and this fallback fail (e.g. the config directory itself is unwritable) does the plugin give up and not persist the value at all. Windows and macOS almost always have a working OS-level backend, so this mostly matters on Linux.

Send Test Email — verifies your SMTP settings actually work without waiting on any real 30+ minute operation. Fills in the same fields above, then click it: sends one email immediately with a fixed test subject/body, using the exact same code path every "Email me when done" checkbox uses. Runs in a background thread (a misconfigured host/port can hang for up to the 30-second SMTP timeout rather than fail instantly) so napari stays responsive while it tries. Reports "Sent to … — check your inbox" or the specific SMTP error (wrong password, unreachable host, etc.) right there — the fastest way to confirm setup before relying on it for a long run.

Why Tab 4's training notifications still work even if napari is closed the whole time: unlike the other five tools (which run in a background thread inside napari itself, so they need napari to stay open the whole time regardless of email), a launched training run's notification isn't sent by the GUI's live polling — instead, when its checkbox is ticked, the launched background process itself is a small wrapper that runs the real training command, waits for it to finish, *then* sends the email, before exiting. That wrapper is what's detached and survives napari closing, exactly like the training script itself, so the email still arrives on schedule whether or not you ever reopen napari to see it. Clicking **Stop Training** kills the whole thing (wrapper included) before it reaches the email step, so a manual stop doesn't send one — only unattended completions/crashes do.

9i. Calibrate Correct-Label Contrast (from Cellpose-SAM)

Finds the lower-contrast value Correct Label (Tab 2/3) should start from — automatically, instead of by dragging the contrast slider until a silhouette "looks right" by eye. Unlike every other Tab 5 sweep, this one is **not** checked against independent ground truth — it finds whichever value best **reproduces what Cellpose-SAM has already segmented**, since that's the actual question Correct Label needs answered before a user nudges it further for one specific problem cell.

How it works: picks the **Cells** most morphologically complex cells (same skeleton-branch-count "Complexity" measure Resort Labels uses) whose centroid sits at least **Edge margin (µm)** away from the volume's own outer boundary — a proxy for "not close to skin", since a cell right at the boundary is exactly the one most likely to already carry a skin-residue artifact merged in (the thing this calibration should be scored *against*, not accidentally learn from). For each selected cell it samples up to **Slices/cell** Z-slices (default 5 cells × 10 slices = **50 samples**, not 10 total), spread across the middle of that cell's own Z-extent. It then sweeps **Sweep steps** candidate lower-contrast values (auto-scaled to the real intensity range around the samples, not a hardcoded guess) and keeps whichever value best reproduces the most existing 2D footprints, jointly across all samples (mean IoU) — not each sample's own independent best, which would let one outlier pull the result around.

Cellpose-SAM labels layer / **Signal layer** — pick the Labels layer to calibrate against and the Image layer whose contrast should be set (both populated automatically from whatever layers are open).

Cells / **Slices/cell** — default 5 / 10 (50 samples total). **Edge margin (µm)** — default 50.0. **Sweep steps** — default 40 candidate values. **Bbox padding (px)** — default 15, matching Correct Label's own default.

Click **Run Contrast Calibration Sweep**. On success, the report below shows every candidate tried and its mean IoU, and the winning value is **applied directly** to the chosen signal layer: contrast limits are set to `[best lo, best lo + 20]` — no manual step needed afterward.

---

10. Output files and folder structure

All files saved by the plugin go into a dedicated folder named after your original input file:

```
/path/to/your/data/
├── NT54_ch1.ims                          ← original input file
└── NT54_ch1/                             ← output folder (created
automatically)
    ├── NT54_ch1_brain_mask.tif           ← binary brain mask
(0/255, uint8)
    ├── NT54_ch1_brain_only_NoBG.tif      ← brain only, background
removed globally
    ├── NT54_ch1_labels.tif               ← cell labels (int32)
    └── NT54_ch1_statistics.csv           ← per-label statistics
```

The folder is created the first time a file is saved. If no input file has been opened (e.g. you loaded a layer directly in napari), files are saved in the current working directory.

Brain-only suffixes depending on background mode:

Mode	Suffix
Off	*(none)*
1 — Exterior Removed	`_ExtRm`
2 — No Background	`_NoBG`
3 — Random Fill	`_RndFill`

---

11. Statistics CSV — all columns explained

The CSV produced by Generate Statistics has one row per label, with up to 52 columns. 47 are always present; the remaining columns appear only when the corresponding optional feature is enabled. This section explains what each column *means*; for the algorithm/formula/library behind each one, see the separate STATISTICS_GUIDE.md.

Identification

Column	Type	Description
`label`	integer	Label number matching the napari Labels layer (1, 2, 3, …)

Volume

Column	Type	Description
`volume_vox`	integer	Number of voxels belonging to this label
`volume_um3`	float (µm³)	Physical volume in cubic micrometres. Computed as `volume_vox × Z_size × Y_size × X_size`. A typical zebrafish microglia is 1,000–10,000 µm³.
`is_volume_outlier`	boolean	`True` if `volume_vox` falls outside the two GT-tracked bounds — bigger than the largest cell any GT fish has ever confirmed real, or smaller than the smallest one (Common Settings' Min volume floor). See This is verified ground truth above for how those bounds are measured. Flags for human review; never deletes anything.

Position (centroid)

The centroid is the 3D centre of mass of the label — the average position of all its voxels.

Column	Type	Description
`centroid_z_vox`	float	Z position in voxel units
`centroid_y_vox`	float	Y position in voxel units
`centroid_x_vox`	float	X position in voxel units
`centroid_z_um`	float (µm)	Z position in micrometres
`centroid_y_um`	float (µm)	Y position in micrometres
`centroid_x_um`	float (µm)	X position in micrometres

Bounding box

The smallest rectangular box (aligned with the axes) that completely contains the label.

Column	Type	Description
`bbox_dz_um`	float (µm)	Height of the bounding box in Z (depth of the cell in the axial direction)
`bbox_dy_um`	float (µm)	Height of the bounding box in Y
`bbox_dx_um`	float (µm)	Width of the bounding box in X

Size and shape

Column	Type	Description
`eq_diam_um`	float (µm)	**Equivalent sphere diameter** — the diameter of a perfect sphere with the same volume as this label. Formula: `(6V/π)^(1/3)`. Useful as a single "size" number regardless of shape.
`axis1_um`	float (µm)	**Longest principal axis** — the maximum extent of the label along its longest geometric

Column	Type	Description
		direction. Derived from the inertia tensor eigenvectors.
`axis2_um`	float (µm)	**Middle principal axis** — approximated as the average of axis1 and axis3.
`axis3_um`	float (µm)	**Shortest principal axis** — the minimum extent perpendicular to the longest axis.
`elongation`	float	**Elongation ratio** = `axis1 / axis3`. A perfect sphere = 1.0. A cigar-shaped cell = 3.0 or more. The higher the number, the more stretched out the cell is.
`principal_axis_dir`	string	The anatomical direction of the longest axis: `"Z"` (axial), `"Y"` (coronal), or `"X"` (sagittal). Tells you which direction the cell is elongated in.

Surface and compactness

Column	Type	Description
`solidity`	float (0–1)	**Solidity** = `volume / convex_hull_volume`. The convex hull is the smallest convex shape enclosing the label (like shrink-wrap). A solid, convex cell = 1.0. A lobulated or branchy cell with lots of indentations < 1.0. Typical range for microglia: 0.5–0.9.
`extent`	float (0–1)	**Extent** = `volume / bounding_box_volume`. How much of the bounding box is actually filled. A cube = 1.0. A sphere ≈ 0.52. Highly branched cells = much lower.

Column	Type	Description
`surface_area_um2`	float (μm²)	**Surface area** in square micrometres, computed using marching cubes — a 3D mesh is generated from the label boundary and the triangle areas summed. A cell with long thin branches has a much larger surface area than a smooth sphere of the same volume.
`sphericity`	float (0–1)	**Sphericity** = `π^(1/3) × (6V)^(2/3) / A` where V = volume and A = surface area. A perfect sphere = 1.0. Anything less than 1.0 is less spherical. Microglia: typically 0.3–0.8 depending on branch complexity.
`surface_to_volume_ratio`	float (μm⁻¹)	**Surface-to-volume ratio** = surface_area / volume. Higher values indicate more complex, surface-rich morphology relative to cell size. Branches and protrusions increase this dramatically.

Skeleton (branching structure)

These columns require the `skan` package. If `skan` is not installed, they will be 0.

The algorithm skeletonizes the label (reduces it to a 1-voxel-wide skeleton) and analyses the resulting graph of branches.

Column	Type	Description
`n_branches`	integer	Number of skeleton branches. A sphere = 1 branch. A microglia with 4 protrusions = roughly 4–8 branches depending on how they connect.

Column	Type	Description
`n_endpoints`	integer	Number of free-end branch tips (branches that don't loop back). Corresponds roughly to the number of protrusion tips.
`mean_branch_len_um`	float (µm)	Average path length of all skeleton branches in micrometres.
`max_branch_len_um`	float (µm)	Length of the longest individual branch — an indicator of maximum protrusion reach.
`branch_tortuosity`	float (≥1)	Average ratio of path length to straight-line distance per branch. A value of 1.0 = perfectly straight branches. Higher values = winding, curved protrusions.
`branch_density`	float (per 10^6 μm^3)	Number of branches per million cubic micrometres of cell volume. Allows fair comparison between cells of different sizes.
`endpoint_density`	float (per 10^6 μm^3)	Number of branch tips per million cubic micrometres. A proxy for protrusion count normalised by cell volume.
`process_complexity`	float	Combined measure of branching complexity: `n_branches × mean_branch_len / eq_diam`. High values = many long branches relative to cell diameter.

Morphotype classification

Column	Type	Description
`morphotype`	string	Automatic shape classification based on elongation, sphericity, solidity, branch count, and surface-to-volume ratio.

Column	Type	Description
		Categories: **Rod-shaped** (elongated, few branches), **Amoeboid** (round, compact, few branches), **Ramified** (many long branches, low sphericity), **Intermediate-ramified** (moderate branching), **Intermediate** (doesn't fit the above).

Spatial relationships

These columns use all cell centroids together to compute neighbourhood statistics.

Column	Type	Description
`nearest_neighbor_dist_um`	float (µm)	Distance to the closest other cell centroid. Small values = cells are tightly packed; large values = isolated cells.
`nearest_neighbor_ratio`	float	**Clark-Evans 3D index** for this cell: the ratio of its nearest-neighbour distance to the expected distance if cells were randomly distributed at the same density. Values < 1 = clustering; > 1 = regularity/ dispersion.
`local_density_100um`	float (cells/ 10^6 µm³)	Number of other cells within a 100 µm radius sphere, normalised by sphere volume. A measure of local neighbourhood crowding.
`depth_normalized`	float (0–1)	Z position normalised to the full depth range of all cells: 0 = shallowest cell, 1 = deepest. Useful for

Column	Type	Description
		comparing dorsal vs. ventral distribution across samples.

Intensity statistics *(optional — requires Image layer selection)*

Column	Type	Description
`mean_intensity`	float	Mean pixel intensity inside the label mask. Reflects overall fluorescence brightness of the cell.
`integrated_intensity`	float	Sum of all pixel values inside the label (mean × voxel count). Proportional to total fluorescent material in the cell regardless of size.
`intensity_cv`	float (0–∞)	Coefficient of variation of pixel intensities = std / mean. 0 = perfectly uniform. High values = heterogeneous staining, possibly indicating internal structure or imaging artefacts.

Brain region assignment *(optional — requires Shapes layer with boundary lines)*

Column	Type	Description
`brain_region`	string	Name of the anatomical region this cell belongs to (as defined by the boundary lines and region names you provided).
`region_boundary_dist_um`	float (µm)	Distance from this cell's centroid to the nearest region boundary line, in micrometres. Cells near

Column	Type	Description
		boundaries may have mixed characteristics.

Description

Column	Type	Description
`description`	string	A plain-language sentence summarising the cell's shape, generated by the selected description backend. Example (rule-based): *"Label 3: Elongated along Y-axis (2.8:1), volume 4,521 µm³, centroid Z=87.3 Y=142.1 X=203.5 µm. Lobulated/ irregular surface, sphericity 0.41, solidity 0.72. Morphotype: Intermediate-ramified. 6 branches, 4 endpoints (mean 8.3 µm), tortuosity 1.4."*

12. Setting up description backends

Rule-based (offline) — no setup needed

The default. Descriptions are generated using built-in templates based on the numeric values. No internet connection, no API key, no external software. Always available.

Ollama (local, free)

Ollama runs a large language model locally on your machine. No data is sent to external servers, and there is no ongoing cost after the initial download.

Step 1 — Install Ollama

Go to https://ollama.com/download and download the installer for your operating system. Run it.

- On Linux: `curl -fsSL https://ollama.com/install.sh | sh`
- On Mac: Download the `.dmg` and drag to Applications.
- On Windows: Download the `.exe` installer.

Step 2 — Download a model

Open a terminal and run:

```
ollama pull llama3
```

This downloads the Llama 3 model (~4.7 GB). You only need to do this once. Other models you can use:

```
ollama pull mistral      # ~4 GB, fast
ollama pull phi3         # ~2 GB, smaller and faster
ollama pull llama3:70b   # ~40 GB, highest quality — needs 64 GB+
        RAM
```

Step 3 — Verify Ollama is running

Ollama starts automatically in the background after installation. You can confirm it is running:

```
ollama list   # should show your downloaded models
```

Step 4 — Configure in the plugin

In **Tab 3 — Statistics**:

1. Select **Ollama (local, free)** from the Description dropdown.
2. **Endpoint:** leave as `http://localhost:11434` (default). Only change this if Ollama runs on a different machine on your network.
3. **Model:** type the model name you downloaded, e.g. `llama3`.
4. Click **Generate Statistics**.

> If you get an `[Ollama error: ...]` in the CSV description column, check that Ollama is running (`ollama list`) and that the model name matches exactly what you downloaded.

OpenAI API (paid)

OpenAI's GPT models run on OpenAI's servers. You pay per token processed. For statistics descriptions (short prompts, short responses), the cost is very low — roughly $0.001–0.01 per 100 cells with `gpt-4o-mini`.

Step 1 — Create an OpenAI account

Go to https://platform.openai.com and sign up. You will need to provide a credit card for billing.

Step 2 — Generate an API key

1. Log in to https://platform.openai.com.
2. Click your profile icon (top right) → **API keys**.
3. Click + **Create new secret key**.
4. Give it a name (e.g. "napari-zf-microglia-ai").
5. Copy the key immediately — it starts with `sk-` and you can only see it once.

Step 3 — Configure in the plugin

In **Tab 3 — Statistics**:

1. Select **OpenAI API (paid)** from the Description dropdown.
2. **API Key:** paste your `sk-...` key.
3. **Model:** `gpt-4o-mini` (recommended — low cost, good quality). Other options:
   - `gpt-4o` — highest quality, higher cost
   - `gpt-3.5-turbo` — fastest, cheapest, lower quality
4. **Base URL:** leave blank unless you use an OpenAI-compatible proxy.
5. Click **Generate Statistics**.

> The API key is **saved encrypted in your OS's credential store** (Windows Credential Manager / macOS Keychain / Linux Secret Service, via the `keyring` package — not the plugin's own plaintext `config.json`) and prefilled next session, so you only need to paste it once. This key is tied directly to your OpenAI billing, with no separate "app-scoped" variant the way a Gmail App Password is — worth being deliberate about, and revoking/regenerating it from OpenAI's dashboard if you're ever unsure. On Linux specifically, the OS store needs an unlocked Secret Service session (GNOME Keyring/KWallet) — if none is available (e.g. an SSH-only session, or before your first desktop login), the plugin

> automatically falls back to a local Fernet-encrypted file instead of writing the key in plaintext — see Section 9h for exactly what that means and its weaker guarantee compared to the OS store.

Claude API (paid)

Anthropic's Claude models. Similar pricing model to OpenAI. Claude Haiku is very fast and inexpensive.

Step 1 — Create an Anthropic account

Go to https://console.anthropic.com and sign up with a credit card.

Step 2 — Generate an API key

1. Log in to https://console.anthropic.com.
2. Click **API Keys** in the left sidebar.
3. Click + **Create Key**.
4. Give it a name and copy the key (starts with `sk-ant-`).

Step 3 — Configure in the plugin

In **Tab 3 — Statistics**:

1. Select **Claude API (paid)** from the Description dropdown.
2. **API Key:** paste your `sk-ant-...` key.
3. **Model:** `claude-haiku-4-5-20251001` (recommended — fast and cheap). Other options:
   - `claude-sonnet-4-6` — higher quality, moderate cost
   - `claude-opus-4-6` — highest quality, highest cost
4. **Base URL:** leave blank (not used for Claude).
5. Click **Generate Statistics**.

12a. Setting up email notification (Gmail App Password)

The **Email notification** panel in **Tab 5 — Sweeps & Utilities** (Section 9h) configures the credentials behind every "Email me when done" checkbox in the plugin. It works with any SMTP-over-SSL provider, but Gmail is the easiest and free path — this walks through it end to end. If you'd rather use a different provider (Outlook/Office365, a work email server, etc.), skip to **Using a non-Gmail provider** at the bottom.

Step 1 — Turn on 2-Step Verification (if not already on)

App Passwords only exist for Google accounts with 2-Step Verification enabled — this is a Google requirement, not something the plugin asks for.

1. Go to https://myaccount.google.com/security.
2. Under "How you sign in to Google," click **2-Step Verification**.
3. Follow the prompts to turn it on (usually a phone number + a code sent by SMS or the Google Authenticator app).

If it's already on, skip straight to Step 2.

Step 2 — Generate an App Password

1. Go to https://myaccount.google.com/apppasswords (you may be asked to sign in again).
2. Under "App name," type something recognizable, e.g. `napari-zf-microglia-ai`.
3. Click **Create**.
4. Google shows a 16-character password (four groups of four letters, e.g. `abcd efgh ijkl mnop`). Copy it now — this is the only time it's shown. Spaces don't matter; you can paste it with or without them.

This is **not your normal Gmail password** and won't be accepted as one — Google deliberately issues a separate, revocable password for exactly this kind of use (a third-party app sending mail on your behalf). You can revoke it any time from the same App Passwords page without affecting your main account password.

Step 3 — Configure in the plugin

In **Tab 5 — Sweeps & Utilities**, General category, the **Email notification (optional)** panel:

1. **Notify email:** the address that should receive the notification — typically your own Gmail address, but it can be any address you want the report sent to.
2. **SMTP server:** leave as `smtp.gmail.com` (the default).
3. **port:** leave as `465` (the default).
4. **SMTP username:** your full Gmail address (e.g. `you@gmail.com`).
5. **SMTP password:** paste the 16-character App Password from Step 2 — *not* your normal Gmail password.
6. Tick the **"Email me when done"** checkbox (or, for training, **"Email me when this training run stops"**) next to whichever tool you want notified about — see the list in Section 9h. You should get one email the next time that run stops (finishes, crashes, or gets early-stopped).

> **All four fields are saved between sessions; the password specifically is saved encrypted in your OS's credential store** (Windows Credential Manager / macOS Keychain / Linux Secret Service via `keyring`), not the plugin's own plaintext `config.json` — set this up once and it works for every checkbox from then on, no re-entering per session. An App Password is a separate, revocable credential Google issues specifically for this kind of unattended use, not your real account password. (The Statistics tab's OpenAI/Claude API keys, Section 12, use the same encrypted storage now, but those are billing-linked with no equivalent "app-scoped" variant, so they're a somewhat different risk — see the note there.) On Linux, this needs an unlocked Secret Service session (GNOME Keyring/KWallet) to actually persist — if unavailable, the password still works for the current session, it just won't be remembered next time; see Section 9h for the fallback behavior in detail.

Using a non-Gmail provider

Any SMTP server that supports SSL on a fixed port works the same way — just change **SMTP server**/**port** to match your provider and use whatever credentials it issues (an app-specific password if the provider offers one, same reasoning as Gmail's). A few examples:

Provider	SMTP server	Port
Gmail	`smtp.gmail.com`	465
Outlook / Office 365 (personal)	`smtp.office365.com`	587 *(see note below)*
Yahoo Mail	`smtp.mail.yahoo.com`	465

> Note: the plugin's supervisor script always connects via `SMTP_SSL` (implicit TLS from the first byte, no STARTTLS handshake) — this matches Gmail and Yahoo's port-465 behavior. Providers that only offer STARTTLS on port 587 (like Office 365) are not currently supported without a small code change; Gmail is the tested, recommended path.

---

13. Full workflow: from raw stack to labelled cells

Step 1 — Open your file

1. Open the plugin (Plugins → ZF-Microglia-ToolKit (ZF-Microglia-AI)) in napari.
2. Click **Open TIF / IMS file** and select your confocal stack.
3. All channels appear as layers.
4. **Click the microglia channel** (usually ch1, green) in the Layers panel.

Step 2 — Run skin removal

Set these values in Tab 1:

Setting	Value
MONAI Threshold	**0.25**
Erosion	0 (default)
Background	**Option 1** if you plan to use Cellpose-SAM in Step 3, **Option 2** if you plan to use the Pixel Classifier
BG Threshold	**1.40**

Click **Run Skin-Remover** and wait.

What you should see: A `brain_only` layer. With Option 2, microglia appear as bright isolated blobs on a black background, with clear space between cells (needed for the Pixel Classifier). With Option 1, the brain interior is left intact — only the tissue outside the brain is removed (needed for Cellpose-SAM).

If blobs look hollow or have large halos (Option 2): Lower BG Threshold (e.g. 0.40).

If too much dim signal remains between cells (Option 2): Raise BG Threshold (e.g. 0.80).

Step 3 — Create labels

Click the `brain_only` layer Tab 1 just produced (`_ExtRm` or `_NoBG`, matching your choice above) in the Layers panel, then switch to the **Create Labels** tab. It automatically shows the matching section — see Section 6a for the full logic.

Option A — Pixel Classifier (active layer ends in `_NoBG`)

Set these values:

Setting	Value
Smooth σ XY	**1.5**
Smooth σ Z	**3.0**
Min overlap	10% (default)
Min volume	7500 (default)
Min hole size	0 (default — leave unless you have seen real holes disappearing)

Click **Create Labels**.

What you should see: A labels layer where each cell is a different colour. The console prints how many were found.

Tuning: - Too many tiny fragments → increase Min volume or increase both σ values - Two cells merged together → try Split Label (see below) - Cells cut across slices → decrease Min overlap or increase σ Z

Option B — Cellpose-SAM Segmentation (active layer ends in `_ExtRm`)

1. Browse to your Cellpose-SAM checkpoint (Section 3) if not already set.
2. Leave the defaults (Cellprob -2.5, Flow 0.4, Safe-merge max gap 2, Safe-merge min contact 10, Large-contact merge 20) unless you know you need to adjust them — see Section 6c for what each one does.
3. Click **Run Cellpose-SAM Segmentation** and wait — this can take hours for a full-size fish. Progress is shown in the status bar; napari stays usable while it runs.

What you should see: A labels layer where each cell is a different colour, exactly as with the Pixel Classifier.

Step 4 — Review and edit labels in napari

- Toggle the labels layer on/off to compare with the original
- Hover over cells to see their label number
- Zoom through Z slices to verify cells are correctly separated

---

Step 5 — Split merged cells (if needed)

If two cells were labelled as one because they touch:

1. Hover over the merged blob and note its label number (shown in the napari status bar at the bottom).
2. In Tab 2, under **Split Label**:
   - **Target label:** enter the label number (or click the blob and click **Use selected**).
   - **Split into:** 2 (or however many cells are merged).
   - **Smooth σ:** 1.0 (default).
   - **Min distance:** 5 (default).
3. Click **Split Label**.
4. The two (or more) cells are separated at their thinnest connection point.

Step 6 — Sort labels (optional)

Click **Resort Labels** to renumber cells by size or position. This is helpful for consistent reporting:

- By **Size** (largest = label 1) — most common
- By **Centroid Z/Y/X** — for atlas alignment

Step 7 — Save labels

Click **Save Labels**. A file dialog opens pre-filled with the output folder. Accept or change the name and click Save.

Step 8 — Generate statistics

1. Click the **Statistics** tab (Tab 3).
2. Make sure the Labels layer is selected in napari.
3. Choose your description backend.
4. *(Optional)* Select a fluorescence channel under **Intensity statistics** to add mean/integrated/CV columns.
5. *(Optional — see Step 8a below)* Draw region boundary lines in a Shapes layer, then select it under **Brain regions** and enter the region names (e.g. `Optic tectum, Hindbrain`).
6. Click **Generate Statistics**.
7. The CSV is saved automatically to the output folder.

Step 8a — Assign cells to brain regions (optional)

This lets you label each cell as belonging to the **optic tectum**, the **hindbrain**, or any other anatomical region you define by drawing a dividing line across the image.

Orientation reminder: The fish lies along the **X axis** — head at X = 0, tail at X = max. Y runs top to bottom (0 → 2048). The optic tectum / hindbrain boundary therefore appears as a roughly vertical curve when you look at the XY plane — it runs from the top of the brain (small Y) to the bottom (large Y), at some X position. Everything to the **left** of the curve (smaller X) is the optic tectum; everything to the **right** (larger X) is the hindbrain.

Step-by-step:

1. **Scroll to a representative Z slice** where the optic tectum / hindbrain boundary is most clearly visible. In zebrafish 4dpf, this boundary is typically a recognisable change in cell density roughly at the mid-point of the anterior–posterior (X) axis.

2. **Add a Shapes layer:** In napari, click the + icon in the toolbar and choose **Shapes**, or go to **Layers → Add shapes layer**.

3. **Select the path tool:** In the shapes toolbar (appears when the Shapes layer is active), click the **path** icon (a polyline with multiple vertices). Do **not** use the straight line tool — the optic tectum / hindbrain boundary is curved.

4. **Draw the boundary curve from top to bottom:** Start clicking at the top of the brain (small Y, Y ≈ 0 side) and work downward to the bottom of the brain (large Y). Follow the curved anatomical boundary as you click. Double-click on the last point to finish. You typically need 4–10 click points.
    - The optic tectum (anterior, smaller X) will be on the **left** of your drawn path.
    - The hindbrain (posterior, larger X) will be on the **right**.

    > Tip: zoom in on the XY view where the boundary is clearest. If unsure of the exact position, trace the curve slightly anterior (more to the left). You can always select the path layer, press Delete to remove it, and redraw.

5. *(For three or more regions)* Draw one additional path per boundary, each running top to bottom.
6. **Switch to Tab 3 — Statistics** in the plugin.
7. Under **Brain regions**:
    - **Boundary lines:** select your Shapes layer from the dropdown.
    - **Region names:** type the names separated by commas, anterior to posterior:

      `Optic tectum, Hindbrain`

      For three regions, e.g.: `Forebrain, Optic tectum, Hindbrain`
8. Click **Generate Statistics**.

Result: The CSV will include two extra columns:

Column	Description
`brain_region`	Name of the region each cell belongs to
`region_boundary_dist_um`	Distance in μm from the cell to the nearest boundary line

You can then filter the CSV in Excel or Python by `brain_region` to compare microglia density, morphology, or intensity between the optic tectum and the hindbrain.

---

14. Reinstalling after an update

```
pip uninstall napari-zf-microglia-ai -y
pip install git+https://github.com/CTichy/ZF-Microglia-AI.git
```

Then **fully close and reopen napari**. If napari is running when you reinstall, it uses the old version until restarted.

> **Your model path and settings are preserved** across reinstalls. The config is stored in `~/.config/napari-zf-microglia-ai/config.json`.

---

15. Troubleshooting

napari crashed / was killed — how much did I lose?

Every Labels layer currently in the viewer is auto-saved to `<output folder>/<layer_name>_recovery.tif` every 10 minutes for the whole session, overwriting the previous recovery save each time — independent of Save Labels, and independent of whatever caused the crash. Look for a `*_recovery.tif` file next to your fish's other output files; at most ~10 minutes of manual editing (Correct Label, Correct Adjacent Labels, Split/Join Labels, etc.) should be missing from it.

If napari itself was silently killed (no error dialog, the window just vanishes) rather than crashing with a visible Python traceback, that's almost always the Linux OOM-killer, not a bug in whatever you were doing at the time — confirm with `journalctl -k --since "-1 hour" | grep -i "out of memory"` in a terminal; it will name the exact process and how much memory it had grown to. See `[[workstation_hardware]]`-style guidance in this project's own memory for the established pattern (2026-08-27, 2026-09-04): a single napari session can, under specific conditions, balloon to the entire machine's RAM. As of 2026-09-04 every background-worker QTimer in this plugin properly releases its result once done (previously, every completed operation — Correct Label, segmentation, sweeps, etc. — permanently leaked its own result for the rest of the session), so a repeat of that specific cause is fixed; the recovery file above is the safety net regardless.

`conda env create -f environment.yml` fails on Windows with `Didn't find wheel for cucim-cu12`

Fixed in the current `environment.yml` (`cucim-cu12` is now Linux-only there — it has no Windows wheels at all, since it's a Linux/WSL2-only RAPIDS package, and isn't something a different pip flag or index fixes on native Windows). It only accelerates Tab 3 statistics, which fall back to CPU cleanly without it; Tab 1 and Tab 2 are unaffected.

If you still hit this error:

- You're on an older clone — `git pull` in the repo folder, then retry.
- If the environment partially exists from the earlier failed attempt, remove it first: `conda env remove -n zf-microglia-ai`, then `conda env create -f environment.yml` again.

The plugin does not appear in Plugins menu

- Make sure napari is fully closed and reopened after installation.
- Verify installation: `pip show napari-zf-microglia-ai`

"No model selected" after reinstalling

- Click `[...]` and browse to your `.pth` file.
- Config path: `~/.config/napari-zf-microglia-ai/config.json`

Tab 4 (AI Tools) is showing a red or amber warning banner

This is expected, not an error — see Section 8. The tab is always available regardless of GPU; the banner just tells you what to expect. No CUDA GPU means CPU fallback (days-months for a full training run instead of hours); a GPU under 8GB may still work with a lower `batch_size` (try 2, or even 1) before assuming something else is wrong.

"Launch Training" errors that a script wasn't found

`prepare_data.py`/`train.py`/`train_xzyz.py` ship with the plugin (bundled under `napari_zf_microglia_ai/training_scripts/`, installed as package data), so this shouldn't normally happen. If you want to point at a

locally modified copy instead, override the path in `~/.config/napari-zf-microglia-ai/config.json` — keys `monai_prepare_script_path`, `monai_train_script_path`, `cellpose_train_script_path`.

Processing runs on CPU (very slow)

NVIDIA GPU: Check that PyTorch sees CUDA:

```
python -c "import torch; print(torch.cuda.is_available())"
```

Should print `True`. If `False`, reinstall PyTorch with CUDA support from https://pytorch.org.

Apple Silicon: Check MPS:

```
python -c "import torch; print(torch.backends.mps.is_available())"
```

Should print `True` on M1/M2/M3.

Statistics are slow (CPU only, no GPU batch)

Linux: Install CuPy and cuCIM for GPU-accelerated regionprops:

```
pip install cupy-cuda12x cucim-cu12
```

Replace `cuda12x`/`cu12` with your actual CUDA version if different (e.g. `cuda11x` for CUDA 11). After installing, reopen napari — the console will print `regionprops: cuCIM GPU` when statistics are computed.

Windows: `cucim-cu12` has no native Windows build (RAPIDS/cuCIM is Linux/WSL2-only) — this is expected and not fixable with a different pip command on native Windows. Tab 3 statistics run on a CPU-threaded path instead; Tab 1 inference and Tab 2 GPU labelling are unaffected, since those only need `cupy-cuda12x`, which does support Windows. If GPU-accelerated statistics specifically matter to you, run the plugin inside WSL2 (Windows Subsystem for Linux) instead, where the Linux install path applies.

`brain_only` layer looks mostly empty (all black)

BG Threshold is too high — lower it (e.g. from 1.40 toward 0.50-0.60).

Create Labels finds 0 or too few objects

- Lower **Min volume** (try 5000).
- Increase **σ XY** and **σ Z** slightly.
- Make sure you selected the `brain_only` layer (not the raw channel) before clicking Create Labels.

Create Labels finds hundreds of tiny fragments

- Increase **Min volume** to 10000.
- Ensure Option 2 with sufficient BG Threshold was used — the brain_only layer must have clean gaps between cells.

Two cells appear as one label (merged)

- Use **Split Label** (Section 6 above) to separate them at the thinnest neck.
- Or decrease σ XY and rerun Create Labels.

Split Label error: "Only N sub-volume(s) found"

The blob doesn't have a clear separation into the requested number of parts.

- Reduce **Smooth σ** (the distance field is over-smoothed and the saddle disappears).
- Reduce **Min distance** (the two centres are being rejected as too close).
- Check the blob is genuinely two distinct cells — zoom in and inspect it slice by slice.

Ollama description shows `[Ollama error: ...]`

- Verify Ollama is running: open a terminal and run `ollama list`.
- If not running, start it: `ollama serve`
- Check the model name matches exactly: `ollama list` shows available models.
- Default endpoint `http://localhost:11434` — change only if Ollama is on a different machine.

OpenAI/Claude API returns an error

- The API key is saved encrypted (OS credential store, or a local encrypted-file fallback on Linux if that's unavailable — see Section 9h) and prefilled from last session — double check it's still valid (a regenerated/revoked key won't match what's saved).
- Check your account has billing set up and enough credit.
- The model name must match exactly (e.g. `gpt-4o-mini`, `claude-haiku-4-5-20251001`).

"Run Cellpose-SAM Segmentation" errors with `No module named 'cellpose'`

Install it in your environment: `pip install cellpose`. Already included in `environment.yml`/`environment-mac.yml` for fresh installs — if you set up your environment before this feature was added, run `conda env update --name zf-microglia-ai -f environment.yml --prune` (or `environment-mac.yml`) and reinstall.

Neither Pixel Classifier nor Cellpose-SAM section shows up in Tab 2

The active layer's name must end in `_ExtRm` (Cellpose-SAM) or `_NoBG` (Pixel Classifier) — reselect the correct Tab 1 output layer in the Layers panel. See Section 6a.

Quick Reference Card

Tab 1 — Skin Remover

Control	Recommended	What it does
MONAI Threshold	0.25	AI confidence cutoff
Erosion	0	Strips voxels from mask edge
Background	Option 2	Removes background globally (best for labels)
BG Threshold	1.40	

Control	Recommended	What it does
		Fine-tunes background removal level
Email me when done	Unchecked	Uses Tab 5's shared Email notification credentials (Section 9h) — useful on CPU/MPS, where this can run 30-60 min

Tab 2 — Create Labels

Shown automatically based on active layer suffix — `_ExtRm` → Cellpose-SAM, `_NoBG` → Pixel Classifier (see 6a).

Common Settings (always visible, above both routes)

Control	Recommended	What it does
Min volume	7500 (until a Tab 5 sweep or GT-verified Statistics run measures a real recommendation)	**Informative only, not editable** — Pixel Classifier's cutoff **and** Cellpose-SAM's Safe-merge "already a whole cell" floor, one shared value; only moves via a GT sweep/GT-verified Statistics run
Max volume	not yet measured (until a GT-verified Statistics run measures one)	**Informative only, not editable** — largest cell ever confirmed real by GT; drives no pipeline stage, only flags `is_volume_outlier` (Tab 3 CSV)
Min hole size	0 (until a GT sweep or GT-verified Statistics run measures a real recommendation)	Editable slider — shared by both routes — minimum voxels for an enclosed gap to survive as real background instead of being filled
Final min-size fraction	0.618 (golden ratio)	Editable slider — both routes — the real

Control	Recommended	What it does
		deletion cutoff is this fraction of Min volume, not Min volume itself (Cellpose-SAM: last-stage safety net; Pixel Classifier: the volume filter's own cutoff)
Soften label contours (sanding)	Checked	Shared by Correct Label, Correct Adjacent Labels, and Cellpose-SAM auto-correct — sands the touched label(s) right after each, foreign-protected, purely geometric
Sanding sigma XY / Z	0.7 / 0.7 vox	Blur strength for sanding — deliberately much smaller than Pixel Classifier's Smooth σ XY/ Z below (different job: polishing an existing label, not merging blobs into one)

Pixel Classifier

Control	Recommended	What it does
Smooth σ XY	1.5	Contour softness within each slice
Smooth σ Z	3.0	Cross-slice blob connectivity
Min overlap	10%	Overlap needed to link blobs across slices

Cellpose-SAM Segmentation

Control	Recommended	What it does
Min size	15 vox	Cellpose-SAM only, not shared with Common Settings' Min volume — tiny early noise filter
	-2.5	

Control	Recommended	What it does
Cellprob threshold		Confidence cutoff for foreground vs. background
Safe-merge max gap	2 vox	Max gap allowed when merging fragments
Safe-merge min contact	10 vox	Min touching surface required to merge
Large-contact merge	20 vox	Second merge pass for thick-junction splits
Email me when done	Unchecked	Uses Tab 5's shared Email notification credentials (Section 9h) — do_3D can run hours on a full-size fish
Re-run This Cell Only	—	Fixes one label without redoing the whole fish — crops, re-runs do_3D + cleanup on just that label, splices the result back in

Both methods (once labels exist)

Control	Recommended	What it does
Split mode	3D (whole label)	2D restricts the split to the current slice only — for artifacts that only touch on one cross-section
Split σ	1.0	Smoothness for watershed split
Min distance	5	Peak separation for split detection
Join Labels	—	Merges Label B into Label A — the inverse of Split Label
Correct Label	2D mode	Regenerates a label's shape from the signal layer's live contrast window — 2D (current

Control	Recommended	What it does
		slice) or 3D (whole cell, per-slice local areas, joint correction wherever a real neighbor is nearby, auto debris cleanup, reports nearby/touching foreign labels and any slice still touching its own local area's edge) — optional auto-grow retries with a bigger pad if signal reaches the edge (per-slice in 3D), auto-folding in neighbors instead of encroaching; 3D also has an independent "keep re-running until stable" switch that re-seeds the joint watershed from each pass's own result until the shape stops changing
Copy Label to Adjacent Slice	—	Copies a label's shape from the current slice onto the next/previous slice
Correct Adjacent Labels	2D only	Jointly corrects two touching labels on the current slice (rectangle scoped to Label A alone), cut placed by watershed seeded at each label's own existing footprint — same optional auto-grow as Correct Label, seeded with both labels
Protect Skin as Label	Brain mask layer	Fills everything outside the brain mask with a real, protected label (ID -1 by default), then trims it to real signal via the

Control	Recommended	What it does
		same 3D engine — so no other label can ever bleed into skin residue again. Reports which real labels end up touching/nearby skin, worth a closer look

Tab 3 — Statistics

Control	Options	What it does
Description	Rule-based / Ollama / OpenAI / Claude	Engine for the description column
Image layer	Any Image layer / None	Adds intensity statistics (mean, integrated, CV)
Boundary lines	Any Shapes layer / None	Assigns cells to named brain regions
Region names	Comma-separated text	Names for each region (N lines → N+1 names)
Generate Statistics	—	Computes up to 45 metrics per label, saves CSV

Tab 4 — AI Tools

Always shown — a banner at the top warns if your GPU is missing or under the recommended 8GB (see Section 8), but doesn't block anything. Switch below it picks MONAI or Cellpose-SAM training.

Control	Default	What it does
Email me when this training run stops	Unchecked	Per-launcher opt-in — uses the shared credentials from Tab 5's Email notification panel (Section 9h); email still arrives if napari never reopens
n_val / n_test	5 / 5	(MONAI) fish held out for val/test in Prepare Training Data
epochs	1500	(MONAI) training length
n_epochs	200	(Cellpose-SAM) training length

Control	Default	What it does
branch_weight	0	(Cellpose-SAM) 0 = standard loss; >0 weights thin/branch pixels more heavily
branch_radius	3 px	(Cellpose-SAM) erosion-survival distance threshold for the branch-weighted loss — measurable from real GT via Calibrate branch_radius below
Calibrate branch_radius (from GT)	—	(Cellpose-SAM) measures real branch thickness from a GT labels volume (3D skeleton + distance transform) — **recommendation auto-applied to branch_radius and saved**
pretrained	Tab 2's checkpoint	(Cellpose-SAM) starting point — "continue training" by default
Extract XZYZ Patches	crop_size=512	(Cellpose-SAM) generates training crops in 3 orientations, cleans truncated labels by default
Patience (checkpoints)	5	Both — stop after N checkpoints with no improvement (Dice/ test_loss); 0 disables
Launch Training	—	Starts a detached process that survives closing napari; GUI reconnects automatically next time
(on stop, Cellpose-SAM only)	—	Writes `<model_name>_best_recommended.txt` in `models/` — a pointer, not a copy, to the best-test_loss checkpoint
Stop Training	—	Kills the training process and its children

Tab 5 — Sweeps & Utilities

Nine tools consolidated from Tabs 1-4, each individually collapsible — see Section 9 for full detail. Every row reads from/writes back to the tab noted in parentheses. A "Show tools for…" filter at the top of the tab (4 checkboxes: Skin Removal / Pixel Classifier / Cellpose-SAM / General, all on by default) hides whichever categories you don't need.

Control	Scope	What it does
Verify MONAI Threshold / Erosion (GT Sweep)	5x5 grid	(Tab 1) Confirms current values against a hand-corrected GT brain mask — MONAI runs once, rest is cheap — **cross-fish average auto-applied to the sliders and saved**
Verify BG Threshold / Erosion (GT Sweep)	5x5 grid	(Tab 1/2) Confirms current BG Threshold/Erosion against real GT IoU, and measures Min volume + Min hole size as never-rising floors from GT — CPU-OK, doesn't survive closing napari — **cross-fish average + Min volume + Min hole size floors auto-applied and saved**
Verify Smooth σ XY / σ Z (GT Sweep)	grid	(Tab 2) Confirms current Smooth σ XY/Z against real GT IoU, BG Threshold/Erosion held fixed — CPU-OK, doesn't survive closing napari — **cross-fish average + Min volume + Min hole size floors auto-applied and saved**
Verify Cellprob / Large-contact (GT Sweep)	5x5 grid, full fish, ~3h total	(Tab 2) Confirms against whole-fish GT — do_3D's network pass runs once for the whole grid (~3h on a full-size fish, GPU-preferred), Cellprob + Large-contact both re-thresholded cheaply on top — **cross-fish average + measured GT-min floor auto-applied to the sliders and saved**. Has an "Email me when done" checkbox (~3h is well past the 30-min mark)
Verify Best Epoch (GT Sweep)	5 cells, ±2 checkpoints	(Tab 4, Cellpose-SAM) confirms the recommendation against real GT IoU/Dice, not just

Control	Scope	What it does
		test_loss — doesn't survive closing napari — **if the sweep disagrees, rewrites the pointer to the confirmed epoch and loads it as Tab 2's active model**. Has an "Email me when done" checkbox (can run 30 min to a couple hours)
Calibrate Correct-Label Contrast (from Cellpose-SAM)	50 samples (5 cells x 10 slices)	(Tab 2/3, Correct Label) finds the lower-contrast value Correct Label should start from by reproducing what Cellpose-SAM already segmented (mean IoU), not independent GT — **auto-applies `[best lo, best lo + 20]` to the chosen signal layer's contrast limits**
Score Against GT	any 2 Labels layers	Whole-fish Hungarian-matched TP/FP/FN/Score/MeanIoU/MeanDice between any two Labels layers — synchronous, no GPU needed
Build GT-Correction Package	—	(Tab 2) Zips the most-advanced correction stage available (sanded > auto-corrected > Krendl-only) + stats CSV + creation guide + optional brain_mask.tif (needed for Protect Skin as Label on the reviewer's end) for external manual correction — browsing Source image auto-fills the rest
Email notification (optional)	*(blank = off)*	Shared SMTP credentials (address, server, port, username, password — password saved **encrypted** via the OS credential store) for every "Email me when done" checkbox in the plugin; configuring it here doesn't

Control	Scope	What it does
		itself send anything — see Section 9h
Send Test Email	—	Sends one email immediately (no GPU, no waiting) to confirm SMTP settings before relying on them for a long run

Plugin developed at FH Technikum Wien — Artificial Intelligence & Data Science Contact: carlos.tichy@gmail.com